

SPIS TREŚCI

1	Wstęp	5
1.1	Cel pracy	5
1.2	Zakres pracy	5
1.3	Problem badawczy	6
1.4	Struktura pracy	7
2	Podstawy teoretyczne	9
2.1	Poker jako gra z niepełną informacją	9
2.2	Texas Hold'em	9
2.3	Ranking układów pokerowych	9
2.4	Ultimate Texas Hold'em	10
2.4.1	Charakterystyka gry	10
2.4.2	Różnice względem klasycznego Texas Hold'em	10
2.4.3	Przebieg rozdania	11
2.4.4	Zakłady i decyzje gracza	11
2.4.5	Rozstrzyganie rozdania	11
2.4.6	Zakłady poboczne	12
2.4.7	Tabele wypłat	12
2.5	Wartość oczekiwana, house edge i strategia bazowa	13
2.6	Uczenie przez wzmacnianie	14
3	Projekt i implementacja środowiska Ultimate Texas Hold'em	17
3.1	Założenia projektowe środowiska	17
3.2	Architektura rozwiązania	18
3.2.1	Przepływ danych podczas wykonania akcji	20
3.3	Model kart i sterowanie talią	21
3.3.1	Reprezentacja karty	21
3.3.2	Standardowa talia	21
3.3.3	Abstrakcja źródła kart	22
3.3.4	Deterministyczna talia testowa	22
3.3.5	Powtarzalność losowych rozdań	23
3.4	Ocena układów pokerowych	23

3.4.1	Reprezentacja wartości ręki	23
3.4.2	Ocena układu pięciokartowego	24
3.4.3	Obsługa niskiego strita z asem	24
3.4.4	Wybór najlepszej ręki	25
3.5	Stan wewnętrzny i obserwacja agenta	26
3.5.1	Pełny stan rozdania	26
3.5.2	Obserwacja agenta	27
3.5.3	Projekcja stanu na obserwację	28
3.5.4	Ochrona przed wyciekiem informacji	29
3.6	Maszyna stanów i przestrzeń akcji	30
3.6.1	Pojedynczy zakład Play i walidacja decyzji	32
3.7	Showdown i rozliczanie zakładów	32
3.7.1	Przebieg showdownu	32
3.7.2	Kwalifikacja krupiera	33
3.7.3	Model rozliczenia zakładów	34
3.7.4	Rozliczenie zakładu Blind	35
3.7.5	Rozliczenie Ante i Play	35
3.7.6	Rozliczenie spasowania	35
3.7.7	Przykładowe rozliczenie	36
3.8	Funkcja nagrody i wynik epizodu	36
3.9	Interfejs silnika	37
3.9.1	Tworzenie silnika	37
3.9.2	Rozpoczęcie epizodu	37
3.9.3	Wykonanie decyzji	38
3.9.4	Właściwości publiczne	38
3.9.5	Integracja z Gymnasium	39
3.10	Rejestrowanie przebiegu i walidacja środowiska	39
3.10.1	Opcjonalne rejestrowanie historii	39
3.10.2	Rekord pojedynczej decyzji	40
3.10.3	Ślad całego rozdania	41
3.10.4	Transakcyjne zapisywanie decyzji	41
3.10.5	Testy deterministyczne	42
3.10.6	Poziomy testowania	43
3.10.7	Pokrycie kodu	43
3.11	Ograniczenia środowiska	43
3.11.1	Zakres podstawowego środowiska	44
3.11.2	Pominięcie zakładów dodatkowych	44
3.11.3	Brak modelu salda i sesji	45
3.11.4	Model pojedynczego gracza	45

3.11.5	Ograniczenie do jednej konfiguracji reguł	45
3.11.6	Brak numerycznej reprezentacji obserwacji	46
3.12	Podsumowanie	46

1. Wstęp

Gry karciane stanowią interesujący obszar zastosowań metod sztucznej inteligencji, ponieważ łączą element losowości, niepełnej informacji oraz sekwencyjnego podejmowania decyzji. W przeciwieństwie do wielu klasycznych problemów uczenia maszynowego, w których model uczy się na podstawie gotowego zbioru danych, w grach agent musi samodzielnie podejmować decyzje, obserwować ich konsekwencje i stopniowo poprawiać swoją strategię.

Szczególnie interesującym przypadkiem są gry pokerowe, w których wynik pojedynczego rozdania zależy nie tylko od jakości posiadanych kart, ale również od decyzji podejmowanych w kolejnych etapach gry. Ultimate Texas Hold'em jest kasynową odmianą pokera opartą na zasadach Texas Hold'em, w której gracz rywalizuje z krupierem, a nie z innymi graczami. Gra ta posiada ograniczoną liczbę możliwych akcji, jasno określone reguły wypłat oraz sekwencyjny przebieg rozdania, co czyni ją odpowiednim środowiskiem do modelowania jako problem uczenia przez wzmacnianie.

Uczenie przez wzmacnianie (ang. *reinforcement learning*, RL) pozwala analizować sytuacje, w których agent uczy się strategii poprzez interakcję ze środowiskiem. W przypadku Ultimate Texas Hold'em środowiskiem może być symulator gry, stanem aktualna sytuacja w rozdaniu, akcją decyzja gracza, natomiast nagrodą końcowy wynik finansowy rozdania. Takie podejście umożliwia badanie, w jaki sposób różne reprezentacje stanu oraz różne definicje funkcji nagrody wpływają na jakość wyuczonej strategii.

1.1. Cel pracy

Głównym celem pracy jest porównanie wybranych metod uczenia przez wzmacnianie w zadaniu uczenia strategii gry w Ultimate Texas Hold'em. Szczególny nacisk położony zostanie na analizę wpływu reprezentacji stanu oraz funkcji nagrody na jakość strategii uzyskiwanej przez agenta.

Celem praktycznym pracy jest implementacja środowiska symulującego rozgrywkę w Ultimate Texas Hold'em oraz przygotowanie agentów podejmujących decyzje w kolejnych etapach rozdania. Uzyskane strategie zostaną porównane ze strategiami bazowymi, takimi jak agent losowy oraz prosty agent regułowy.

1.2. Zakres pracy

Zakres pracy obejmuje zaprojektowanie i implementację uproszczonego środowiska gry Ultimate Texas Hold'em, w którym uwzględnione zostaną podstawowe elementy rozgrywki: karty gracza i krupiera, karty wspólne, obowiązkowe zakłady Ante i Blind oraz

decyzyjny zakład Play. W podstawowej wersji środowiska pominięte zostaną zakłady poboczne, takie jak Trips oraz Progressive, ponieważ nie są one bezpośrednio związane z głównym procesem decyzyjnym agenta.

W ramach pracy zostaną przygotowane różne warianty reprezentacji stanu gry. Reprezentacja stanu może obejmować między innymi informacje o kartach gracza, kartach wspólnych, aktualnym etapie rozdania oraz dostępnych akcjach. Analizie podlegać będzie również sposób definiowania funkcji nagrody, w szczególności to, czy agent otrzymuje nagrodę wyłącznie na końcu rozdania, czy także dodatkowe informacje pośrednie wspierające proces uczenia.

Założenia niniejszej pracy są następujące:

1. Implementacja środowiska symulującego rozgrywkę w Ultimate Texas Hold'em zgodnie z podstawowymi zasadami gry.
2. Implementacja mechanizmu oceny układów pokerowych oraz rozstrzygania wyniku rozdania pomiędzy graczem a krupierem.
3. Przygotowanie agentów bazowych, w szczególności agenta losowego oraz prostego agenta regułowego.
4. Implementacja i porównanie wybranych metod uczenia przez wzmacnianie w zadaniu podejmowania decyzji w Ultimate Texas Hold'em.
5. Analiza wpływu różnych reprezentacji stanu na jakość wyuczonej strategii.
6. Analiza wpływu różnych funkcji nagrody na stabilność procesu uczenia oraz końcowy wynik agenta.
7. Ocena agentów z wykorzystaniem metryk takich jak średni wynik na rozdanie, wartość oczekiwana strategii oraz porównanie względem strategii bazowych.

1.3. Problem badawczy

Problem badawczy rozważany w pracy można sformułować następująco: w jaki sposób wybór reprezentacji stanu oraz funkcji nagrody wpływa na skuteczność metod uczenia przez wzmacnianie w grze Ultimate Texas Hold'em?

W pracy analizowane będzie, czy agent uczony metodami reinforcement learning jest w stanie uzyskać strategię lepszą od strategii losowej oraz prostych heurystyk. Ze względu na konstrukcję gier kasynowych celem nie jest wykazanie, że agent może trwale osiągnąć dodatnią wartość oczekiwaną przeciwko kasynu, lecz sprawdzenie, czy możliwe jest nauczenie strategii minimalizującej oczekiwaną stratę.

1.4. Struktura pracy

Praca składa się z kilku rozdziałów. W rozdziale pierwszym przedstawiono cel, zakres oraz problem badawczy pracy. Rozdział drugi zawiera podstawy teoretyczne dotyczące pokera, Texas Hold'em, Ultimate Texas Hold'em, wartości oczekiwanej, przewagi kasyna oraz uczenia przez wzmacnianie.

W kolejnych rozdziałach opisany zostanie projekt środowiska symulacyjnego, sposób reprezentacji stanu gry, funkcja nagrody oraz zastosowane metody uczenia przez wzmacnianie. Następnie przedstawione zostaną eksperymenty, uzyskane wyniki oraz analiza porównawcza agentów. Ostatni rozdział będzie zawierał podsumowanie pracy, wnioski oraz możliwe kierunki dalszego rozwoju.

2. Podstawy teoretyczne

2.1. Poker jako gra z niepełną informacją

Poker jest rodziną gier karcianych, w których wynik zależy zarówno od losowego rozdania kart, jak i od decyzji podejmowanych przez graczy. W przeciwieństwie do gier z pełną informacją, takich jak szachy, uczestnicy rozgrywki nie znają wszystkich elementów stanu gry. Ukryte pozostają przede wszystkim karty przeciwników, co sprawia, że decyzje podejmowane są w warunkach niepewności.

Z punktu widzenia sztucznej inteligencji poker jest interesującym problemem badawczym, ponieważ łączy losowość, niepełną informację, sekwencyjne podejmowanie decyzji oraz konieczność maksymalizacji wyniku w długim horyzoncie czasowym. W takim środowisku pojedyncza decyzja nie powinna być oceniana wyłącznie przez pryzmat wyniku jednego rozdania, lecz przez jej wpływ na oczekiwaną wartość strategii w wielu powtórzeniach gry.

2.2. Texas Hold'em

Texas Hold'em jest jedną z najpopularniejszych odmian pokera. Każdy gracz otrzymuje dwie zakryte karty własne (ang. *hole cards*), natomiast na stole stopniowo odkrywanych jest pięć kart wspólnych (ang. *community cards*). Gracz tworzy najlepszy możliwy układ pięciokartowy, wykorzystując dowolną kombinację swoich dwóch kart oraz pięciu kart wspólnych.

Rozgrywka w klasycznym Texas Hold'em składa się z kilku etapów: etapu przed flopem (ang. *preflop*), flopa (ang. *flop*), czyli odkrycia trzech pierwszych kart wspólnych, turna (ang. *turn*), czyli czwartej karty wspólnej, rivera (ang. *river*), czyli piątej i ostatniej karty wspólnej, oraz showdownu (ang. *showdown*), czyli końcowego porównania układów. W trakcie kolejnych rund licytacji gracze mogą podejmować decyzje takie jak czekanie (ang. *check*), postawienie zakładu (ang. *bet*), sprawdzenie zakładu przeciwnika (ang. *call*), podbicie (ang. *raise*) lub spasowanie (ang. *fold*). W odmianach no-limit możliwe jest również stosowanie różnych rozmiarów zakładów, co istotnie zwiększa przestrzeń możliwych strategii.

2.3. Ranking układów pokerowych

W Texas Hold'em oraz Ultimate Texas Hold'em wynik rozdania zależy od siły najlepszego układu pięciokartowego utworzonego z dostępnych kart. Układy pokerowe mają ustaloną hierarchię, która pozwala jednoznacznie porównać rękę gracza i krupiera

podczas showdownu. Przykładowy ranking układów od najsilniejszego do najslabszego przedstawiono w tabeli 2.1 [1].

Najsilniejszym układem jest poker królewski, natomiast najslabszym układem jest wysoka karta. W przypadku gdy dwóch uczestników rozdania posiada układ tej samej kategorii, o zwycięstwie decydują wartości kart wchodzących w skład układu oraz karty dodatkowe, określane jako kickery.

Tabela 2.1. Ranking układów pokerowych od najsilniejszego do najslabszego

Układ	Opis
Poker królewski	A, K, Q, J, 10 w tym samym kolorze
Poker	Pięć kolejnych kart w tym samym kolorze
Kareta	Cztery karty tej samej wartości
Full	Trójka oraz para
Kolor	Pięć kart w tym samym kolorze
Strit	Pięć kolejnych kart
Trójka	Trzy karty tej samej wartości
Dwie pary	Dwa różne układy par
Para	Dwie karty tej samej wartości
Wysoka karta	Brak silniejszego układu

2.4. Ultimate Texas Hold'em

2.4.1. Charakterystyka gry

Ultimate Texas Hold'em jest kasynową odmianą pokera opartą na zasadach Texas Hold'em. W przeciwieństwie do klasycznej odmiany wieloosobowej gracz nie rywalizuje z innymi graczami, lecz z krupierem reprezentującym kasyno. Zarówno gracz, jak i krupier otrzymują po dwie karty własne, a następnie korzystają z pięciu kart wspólnych w celu utworzenia najlepszego możliwego układu pięciokartowego [1], [2].

2.4.2. Różnice względem klasycznego Texas Hold'em

Najważniejsza różnica względem klasycznego Texas Hold'em polega na tym, że w Ultimate Texas Hold'em gracz podejmuje decyzje przeciwko kasynu, a nie przeciwko innym graczom. Nie występuje więc klasyczna rywalizacja między wieloma uczestnikami stołu, nie ma konieczności blefowania przeciwników ani reagowania na ich indywidualne style gry.

Istotną cechą Ultimate Texas Hold'em jest również ograniczona liczba momentów decyzyjnych. Gracz może wykonać zakład Play (ang. *Play bet*) tylko raz w trakcie rozdania: przed flopem, po flopie albo po odkryciu wszystkich kart wspólnych. Im wcześniej gracz zdecyduje się na zagranie agresywne, tym większy zakład może postawić, ale decyzja jest wtedy podejmowana przy mniejszej liczbie informacji [1], [3].

Tabela 2.2. Porównanie Texas Hold'em i Ultimate Texas Hold'em

Cecha	Texas Hold'em	Ultimate Texas Hold'em
Przeciwnik	inni gracze	krupier / kasyno
Liczba graczy	wielu	jeden gracz przeciwko dealerowi
Zakłady	zmienne	ustalone mnożniki Ante
Blefowanie	istotne	brak klasycznego blefowania
Liczba decyzji	duża	ograniczona

2.4.3. Przebieg rozdania

Rozdanie rozpoczyna się od postawienia przez gracza obowiązkowych zakładów Ante (ang. *Ante bet*) oraz Blind (ang. *Blind bet*). Zakłady te mają zwykle taką samą wartość. Opcjonalnie gracz może również postawić zakład poboczny (ang. *side bet*), na przykład Trips, który jest rozliczany niezależnie od wyniku pojedynku z krupierem [1].

Po wniesieniu zakładów gracz oraz krupier otrzymują po dwie zakryte karty własne. Następnie gracz podejmuje pierwszą decyzję. Może zaczekać albo wykonać zakład Play w wysokości trzykrotności lub czterokrotności zakładu Ante.

Jeżeli gracz nie wykona zakładu Play przed flopem, odkrywane są trzy pierwsze karty wspólne, czyli flop. Na tym etapie gracz może ponownie zaczekać albo wykonać zakład Play w wysokości dwukrotności Ante. Jeżeli również po flopie gracz zdecyduje się zaczekać, odkrywane są dwie ostatnie karty wspólne, czyli turn oraz river. Wtedy gracz musi wybrać pomiędzy spasowaniem a wykonaniem zakładu Play w wysokości jednokrotności Ante [1], [3].

2.4.4. Zakłady i decyzje gracza

W podstawowym modelu gry występują trzy główne rodzaje zakładów: Ante, Blind oraz Play. Zakłady Ante i Blind są obowiązkowe i stawiane przed rozpoczęciem rozdania. Zakład Play jest natomiast bezpośrednio związany z decyzją gracza i może zostać wykonany tylko raz w trakcie rozdania.

Z punktu widzenia modelowania problemu jako środowiska uczenia przez wzmocnienie najważniejszy jest zakład Play, ponieważ jego wysokość i moment wykonania wynikają z decyzji agenta. Ante i Blind można traktować jako początkowy koszt udziału w rozdaniu, natomiast Play odzwierciedla wybór strategii w aktualnym stanie gry.

Dostępne decyzje gracza w poszczególnych etapach rozdania przedstawiono w tabeli 2.3.

2.4.5. Rozstrzygnięcie rozdania

Po zakończeniu etapu decyzyjnego krupier odsłania swoje dwie karty własne. Zarówno gracz, jak i krupier tworzą najlepszy możliwy układ pięciokartowy z siedmiu

Tabela 2.3. Dostępne decyzje gracza w Ultimate Texas Hold'em

Etap	Dostępne akcje	Wysokość zakładu Play
Preflop	check / raise	3x lub 4x Ante
Flop	check / raise	2x Ante
River	fold / raise	1x Ante

dostępnych kart: dwóch kart własnych oraz pięciu kart wspólnych. Następnie układy są porównywane zgodnie ze standardowym rankingiem układów pokerowych.

Krupier kwalifikuje się do gry, jeżeli posiada co najmniej parę. Jeżeli krupier się nie kwalifikuje, zakład Ante jest zwracany graczowi. Zakład Play jest rozliczany na podstawie bezpośredniego porównania układu gracza i krupiera. W przypadku zwycięstwa gracza Play wypłacany jest w stosunku 1:1, w przypadku przegranej gracz traci zakład Play, a w przypadku remisu zakład jest zwracany [1], [3].

Zakład Blind ma odrębną logikę rozliczania. Jeżeli gracz pokona krupiera układem o wartości co najmniej strita, Blind wypłacany jest zgodnie z tabelą wypłat. Jeżeli gracz wygra rozdanie układem słabszym niż strit, zakład Blind jest zwracany. W przypadku przegranej gracza zakład Blind jest tracony, a w przypadku remisu zwracany [1].

2.4.6. Zakłady poboczne

W wielu wariantach Ultimate Texas Hold'em dostępne są również zakłady poboczne, takie jak Trips lub Progressive. Zakład Trips jest opcjonalny i wypłacany na podstawie końcowego układu gracza, niezależnie od tego, czy gracz pokonał krupiera. Najczęściej wygrywa on wtedy, gdy końcowy układ gracza ma wartość co najmniej trójki [1].

Zakłady poboczne mogą różnić się w zależności od kasyna i stosowanej tabeli wypłat.

TODO: Decyzja do potwierdzenia z promotorem: czy w podstawowym środowisku uwzględnić za

W podstawowej wersji środowiska zakłady poboczne Trips oraz Progressive nie będą uwzględniane. Wynika to z faktu, że głównym celem pracy jest modelowanie sekwencyjnego procesu decyzyjnego dotyczącego zakładu Play. Zakłady poboczne są opcjonalne, zależą od tabel wypłat konkretnego kasyna i zwiększają wariancję wyniku, nie zmieniając podstawowej struktury decyzji: check, raise albo fold.

2.4.7. Tabele wypłat

Zakład Blind nie jest wypłacany dla każdego zwycięskiego układu gracza. Zgodnie z tabelą wypłat, bonus na zakładzie Blind wypłacany jest wtedy, gdy gracz pokona krupiera i posiada układ o wartości co najmniej strita. Jeżeli gracz pokona krupiera układem słabszym niż strit, zakład Blind jest zwracany. Tabelę wypłat zakładu Blind

przedstawiono w tabeli 2.4 [1].

Tabela 2.4. Tabela wypłat zakładu Trips przyjęta w modelu środowiska

Układ gracza	Wypłata
Poker królewski	500:1
Poker	50:1
Kareta	10:1
Full	3:1
Kolor	3:2
Strit	1:1
Pozostałe układy	Zwrot

Zakład Trips jest opcjonalnym zakładem pobocznym. Jest on rozliczany na podstawie końcowego układu gracza i nie zależy od tego, czy gracz pokonał krupiera. W analizowanym przykładzie zakład Trips wygrywa wtedy, gdy gracz uzyska układ co najmniej trójki. Przykładową tabelę wypłat zakładu Trips przedstawiono w tabeli 2.5 [1].

Tabela 2.5. Przykładowa tabela wypłat zakładu Trips w Ultimate Texas Hold'em

Układ gracza	Wypłata
Poker królewski	50:1
Poker	40:1
Kareta	30:1
Full	8:1
Kolor	7:1
Strit	4:1
Trójka	3:1

W niektórych wariantach gry dostępny jest również zakład Progressive, którego tabela wypłat zależy od konkretnego kasyna oraz wartości puli progresywnej. Ze względu na zależność od zewnętrznych zasad kasynowych oraz brak bezpośredniego związku z podstawową decyzją Play, zakład ten nie będzie uwzględniany w podstawowym modelu środowiska.

2.5. Wartość oczekiwana, house edge i strategia bazowa

Wartość oczekiwana (ang. *expected value*, EV) określa średni wynik finansowy decyzji lub strategii przy założeniu dużej liczby powtórzeń gry. W kontekście Ultimate Texas Hold'em pojedyncze rozdanie może zakończyć się zarówno zyskiem, jak i stratą, jednak jakość strategii należy oceniać na podstawie średniego wyniku uzyskiwanego w wielu rozdaniach.

Przewaga kasyna (ang. *house edge*) jest matematyczną miarą określającą oczekiwany zysk kasyna względem zakładu gracza. Nie oznacza ona, że kasyno wygrywa każde

pojedyncze rozdanie, lecz że przy dużej liczbie rozegranych rozdań średni wynik będzie przesuwał się na korzyść kasyna. Gracz może więc osiągać krótkoterminowe zyski, jednak w długim okresie wynik gry powinien zbliżać się do wartości oczekiwanej wynikającej z zasad gry [4].

Istotne jest również to, że przewaga kasyna odnosi się do kwoty postawionych zakładów, a nie wyłącznie do początkowego kapitału gracza (ang. *bankroll*). Jeżeli gracz wykonuje wiele zakładów w kolejnych rozdaniach, całkowita kwota zaangażowana w grę może być wielokrotnie większa niż początkowy kapitał. Z tego powodu nawet niewielka procentowa przewaga kasyna może prowadzić do istotnej oczekiwanej straty w długim horyzoncie czasowym [4].

W grach takich jak Ultimate Texas Hold'em interpretacja house edge wymaga dodatkowej ostrożności, ponieważ gracz oprócz zakładów początkowych może zwiększać swoje zaangażowanie poprzez zakład Play. Z tego powodu oprócz klasycznej przewagi kasyna można analizować również miarę określaną jako element ryzyka (ang. *element of risk*), odnoszącą średnią stratę do całkowitej kwoty postawionej w rozdaniu [3], [5].

Przykładowe wartości przewagi kasyna dla wybranych gier przedstawiono w tabeli 2.6 [5].

Tabela 2.6. Przykładowe wartości przewagi kasyna dla wybranych gier kasynowych

Gra	Przewaga kasyna
Blackjack	0,28%
Ultimate Texas Hold'em	2,19%
Ruletka europejska	2,70%
Ruletka amerykańska	5,26%
Slot machines	2–15%

Tabela 2.6 pokazuje, że przewaga kasyna jest cechą konstrukcyjną gier hazardowych, ale jej poziom istotnie różni się pomiędzy grami. Celem niniejszej pracy nie jest wykazanie, że agent może trwale uzyskać dodatnią wartość oczekiwaną w grze zaprojektowanej na korzyść kasyna, lecz sprawdzenie, które metody uczenia przez wzmacnianie są w stanie nauczyć się strategii minimalizującej tę przewagę.

2.6. Uczenie przez wzmacnianie

Uczenie przez wzmacnianie (ang. *reinforcement learning*, RL) jest paradygmatem uczenia maszynowego, w którym agent uczy się podejmowania decyzji poprzez interakcję ze środowiskiem. W każdym kroku agent obserwuje aktualny stan (ang. *state*) środowiska, wybiera akcję (ang. *action*), a następnie otrzymuje nagrodę (ang. *reward*) oraz przechodzi do kolejnego stanu. Celem agenta jest wyuczenie strategii, określanej również jako polityka decyzyjna (ang. *policy*), która maksymalizuje sumę nagród w długim horyzoncie czasowym.

W przypadku Ultimate Texas Hold'em środowiskiem jest symulator gry, stanem jest aktualna sytuacja w rozdaniu, akcją jest decyzja gracza, natomiast nagrodą może być końcowy wynik finansowy rozdania. Takie sformułowanie pozwala traktować grę jako problem sekwencyjnego podejmowania decyzji, w którym agent uczy się kompromisu pomiędzy ryzykiem, ilością dostępnej informacji oraz potencjalną wysokością wypłaty.

Z perspektywy uczenia przez wzmocnianie Ultimate Texas Hold'em jest interesującym środowiskiem, ponieważ decyzje gracza są sekwencyjne i podejmowane przy różnym poziomie dostępnej informacji. Przed flopem gracz zna jedynie swoje dwie karty, ale może wykonać najwyższy zakład Play. Po flopie posiada więcej informacji, lecz maksymalny zakład jest mniejszy. Po odkryciu wszystkich kart wspólnych zna już pełny board, ale może wykonać wyłącznie zakład 1x Ante albo spasować. Agent musi więc nauczyć się kompromisu pomiędzy ryzykiem, ilością informacji oraz potencjalną wypłatą.

3. Projekt i implementacja środowiska Ultimate Texas Hold'em

W niniejszym rozdziale przedstawiono projekt oraz implementację środowiska Ultimate Texas Hold'em wykorzystywanego w dalszej części pracy do uczenia i oceny agentów. Opisane wcześniej zasady gry zostały odwzorowane w postaci deterministycznego i testowalnego silnika, który zarządza przebiegiem rozdania, udostępnia agentowi legalne akcje oraz rozlicza wynik finansowy rozgrywki.

Szczególną uwagę poświęcono zachowaniu niepełnej informacji charakterystycznej dla pokera. Agent otrzymuje wyłącznie informacje, które byłyby dostępne rzeczywistemu graczowi, natomiast karty krupiera, karty spalone oraz kolejność pozostałych kart w talii pozostają elementami wewnętrznego stanu środowiska. Takie rozdzielanie pozwala ograniczyć ryzyko niezamierzonego ujawnienia informacji podczas uczenia modeli.

Implementację podzielono na niezależne moduły odpowiedzialne między innymi za reprezentację kart, ocenę układów, sterowanie przebiegiem rozdania, walidację akcji oraz rozliczanie zakładów. Silnik domenowy nie zależy bezpośrednio od biblioteki Gymnasium. Adapter zgodny z jej interfejsem zostanie utworzony jako osobna warstwa, co umożliwi późniejsze porównywanie różnych reprezentacji stanu i funkcji nagrody bez modyfikowania zasad gry.

3.1. Założenia projektowe środowiska

Podstawową jednostką interakcji ze środowiskiem jest jedno kompletne rozdanie. Rozdanie rozpoczyna się od utworzenia talii i przekazania graczowi oraz krupierowi po dwóch kart własnych, a kończy się spasowaniem gracza albo automatycznym rozstrzygnięciem układów. Z punktu widzenia uczenia przez wzmacnianie pojedyncze rozdanie stanowi jeden epizod.

Agent podejmuje decyzje wyłącznie w trzech etapach: przed flopem, po odkryciu flopa oraz po odkryciu wszystkich pięciu kart wspólnych. W zależności od etapu może czekać, wykonać zakład Play o odpowiednim mnożniku albo spasować. Po wykonaniu zakładu Play środowisko automatycznie odkrywa pozostałe karty, ocenia ręce gracza i krupiera oraz przechodzi do stanu końcowego. W ramach jednego rozdania zakład Play może zostać wykonany tylko raz.

Środowisko modeluje podstawowy wariant Ultimate Texas Hold'em obejmujący zakłady Ante, Blind oraz Play. Zakłady boczne Trips i Progressive zostały pominięte. Nie wpływają one na przestrzeń decyzji podstawowej gry, natomiast zmieniają rozkład wyników finansowych i utrudniają ocenę jakości strategii decyzyjnej agenta. Uzasadnienie tego ograniczenia przedstawiono szerzej w dalszej części rozdziału.

Wartości zakładów wyrażono w jednostkach względnych, gdzie jedna jednostka odpowiada wartości zakładu Ante. Zakłady Ante i Blind mają zatem wartość jednej jednostki, natomiast wartość zakładu Play zależy od momentu jego wykonania i może wynosić od jednej do czterech jednostek. Takie rozwiązanie uniezależnia środowisko od konkretnej waluty i nominalnej wysokości stawki, a także ułatwia porównywanie wyników pomiędzy eksperymentami.

Silnik zwraca szczegółowe rozliczenie poszczególnych zakładów oraz łączny wynik netto rozdania. W podstawowym wariancie środowiska przeznaczonym dla agentów uczących się wynik netto będzie wykorzystywany jako nagroda terminalna. W krokach pośrednich nagroda wynosi zero. Rozdzielenie mechanizmu rozliczania gry od funkcji nagrody umożliwia późniejsze badanie alternatywnych wariantów nagrody bez ingerencji w reguły Ultimate Texas Hold'em.

Istotnym założeniem była również powtarzalność eksperymentów. Standardowa talia może być tasowana przy użyciu zadanego ziarna generatora liczb pseudolosowych, dzięki czemu możliwe jest odtworzenie całej sekwencji rozdań. W testach jednostkowych stosowana jest dodatkowo talia deterministyczna, pozwalająca jednoznacznie określić kolejność dobieranych kart i zweryfikować konkretne przypadki przebiegu oraz rozliczenia rozdania.

3.2. Architektura rozwiązania

Środowisko Ultimate Texas Hold'em zaprojektowano jako zestaw niezależnych modułów domenowych. Każdy moduł odpowiada za jeden wyraźnie określony obszar, taki jak reprezentacja kart, ocena układów, sterowanie przebiegiem rozdania albo rozliczanie zakładów. Centralnym elementem rozwiązania jest klasa `UTHGame`, która koordynuje działanie pozostałych komponentów, ale nie implementuje bezpośrednio wszystkich reguł gry.

Przyjęty podział ogranicza zależności pomiędzy poszczególnymi częściami systemu. Model kart i evaluator układów mogą być rozwijane oraz testowane niezależnie od zasad Ultimate Texas Hold'em. Analogicznie mechanizm rozliczania zakładów nie odpowiada za odkrywanie kart ani za walidację dozwolonych akcji. Dzięki temu zmiana jednego elementu, na przykład reprezentacji obserwacji agenta, nie wymaga modyfikowania sposobu oceny układów lub obliczania wypłat.

Najniższą warstwę rozwiązania stanowią moduły `cards` i `hands`. Pierwszy z nich definiuje karty oraz talię, natomiast drugi odpowiada za ocenę i porównywanie układów pokerowych. Moduły te nie zawierają reguł charakterystycznych dla Ultimate Texas Hold'em i mogą zostać ponownie wykorzystane w innych odmianach pokera.

Warstwę domenową gry tworzy pakiet `uth`. Zawiera on modele stanu, reguły legalności akcji, mechanizm rozdawania kart, sposób rozliczania zakładów oraz logikę sterującą kolejnymi fazami rozdania. Klasa `UTHGame` pełni rolę koordynatora tej war-

stwy. Na podstawie aktualnego stanu i wybranej akcji wywołuje odpowiednie operacje, aktualizuje stan rozdania oraz zwraca rezultat kroku.

Agent nie komunikuje się bezpośrednio z talią, evaluatorem ani pełnym stanem wewnętrznym gry. Otrzymuje obiekt `UTHObservation`, a swoją decyzję przekazuje do metody `step`. Rezultatem wykonania akcji jest obiekt `StepResult`, zawierający kolejną obserwację oraz, w przypadku zakończenia rozdania, jego wynik i szczegółowe rozliczenie. Takie rozwiązanie tworzy jednoznaczną granicę pomiędzy logiką środowiska a implementacją agenta.

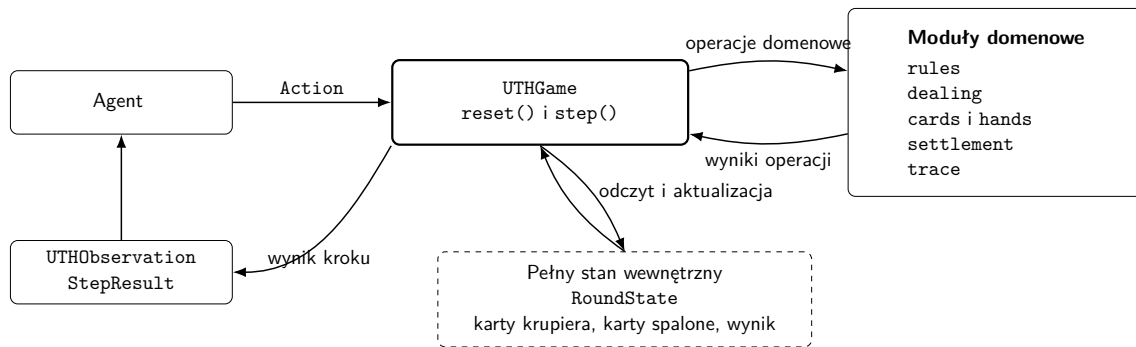
Tabela 3.1 przedstawia najważniejsze moduły rozwiązania i zakres ich odpowiedzialności.

Tabela 3.1. Podział odpowiedzialności pomiędzy modułami środowiska UTH

Moduł	Odpowiedzialność
<code>cards</code>	Reprezentacja rang, kolorów, pojedynczych kart oraz standardowej talii składającej się z 52 unikalnych kart.
<code>hands</code>	Ocena pięciokartowych układów, wybór najlepszego układu z pięciu, sześciu lub siedmiu kart oraz porównywanie wartości rąk.
<code>uth.models</code>	Definicja pełnego stanu rozdania, obserwacji agenta oraz wyniku pojedynczego kroku środowiska.
<code>uth.rules</code>	Określanie zbioru legalnych akcji dla każdej fazy rozdania.
<code>uth.dealing</code>	Realizacja kolejności rozdawania kart, odkrywania flopa, turna i rivera oraz obsługa kart spalonych.
<code>uth.card_source</code>	Abstrakcja źródła kart oraz deterministyczna talia wykorzystywana w testach.
<code>uth.settlement</code>	Określanie wyniku rozdania, kwalifikacji krupiera oraz rozliczanie zakładów Ante, Blind i Play.
<code>uth.game</code>	Koordinacja przebiegu rozdania, walidacja akcji, przechodzenie pomiędzy fazami oraz tworzenie wyników kolejnych kroków.
<code>uth.trace</code>	Opcjonalne rejestrowanie obserwacji, decyzji agenta i końcowego stanu rozdania na potrzeby diagnostyki.

Silnik domenowy nie zależy od konkretnej biblioteki uczenia przez wzmocnianie. Przyszły adapter Gymnasium będzie korzystał z publicznego interfejsu `UTHGame`, przekształcając obiekty domenowe na numeryczne reprezentacje wymagane przez algorytmy uczące się. Pozwala to zachować jedną implementację zasad gry dla wszystkich badanych reprezentacji obserwacji, funkcji nagrody i metod uczenia.

Ogólny przepływ informacji pomiędzy agentem, klasą sterującą i pozostałymi elementami silnika przedstawiono na rysunku 3.1. Agent komunikuje się wyłącznie z publicznym interfejsem klasy `UTHGame`. Pełny stan rozdania oraz moduły realizujące reguły gry pozostają ukryte za tą warstwą.



Rys. 3.1. Architektura i przepływ informacji w środowisku UTH

3.2.1. Przepływ danych podczas wykonania akcji

Po rozpoczęciu rozdania metoda `reset()` zwraca pierwszą obserwację agenta. Na jej podstawie agent wybiera akcję należącą do zbioru akcji legalnych w aktualnej fazie gry. Wybrana wartość jest następnie przekazywana do metody `step()`, która realizuje pojedyncze przejście środowiska.

W pierwszej kolejności silnik sprawdza, czy rozdanie nie zostało już zakończone, czy przekazana wartość jest poprawnym obiektem reprezentującym akcję oraz czy dana akcja jest legalna w aktualnej fazie. Odrzucenie niepoprawnej akcji następuje przed jakąkolwiek zmianą stanu lub pobraniem kart z talii. Zapobiega to częściowemu wykonaniu niedozwolonego przejścia.

Po pozytywnej walidacji sterowanie jest przekazywane do operacji właściwej dla aktualnej fazy: preflop, flopa albo rivera. Akcja oczekiwania powoduje odkrycie kart wymaganych do przejścia do następnej fazy. Wykonanie zakładu Play prowadzi natomiast do odkrycia wszystkich pozostałych kart wspólnych, oceny układów gracza i krupiera, określenia wyniku oraz rozliczenia aktywnych zakładów. Spasowanie na riverze kończy rozdanie bez wykonywania showdownu.

Po pomyślnym przeprowadzeniu operacji tworzony jest nowy obiekt `RoundState`. Na jego podstawie budowana jest obserwacja przeznaczona dla agenta oraz obiekt `StepResult`. Wynik kroku zawiera informację, czy rozdanie zostało zakończone, a w stanie terminalnym również rezultat rozgrywki, szczegółowe rozliczenie zakładów i dane dotyczące showdownu.

Jeżeli rejestrowanie historii jest włączone, obserwacja sprzed wykonania akcji i wybrana decyzja są zapisywane jako `DecisionRecord`. Rejestracja następuje dopiero po prawidłowym zakończeniu przejścia. Jeżeli podczas odkrywania kart lub aktualizacji stanu wystąpi wyjątek, decyzja nie jest dodawana do historii, a poprzedni stan środowiska pozostaje aktualny. Rozwiązanie to zapewnia spójność pomiędzy zapisanym przebiegiem rozdania a rzeczywiście wykonanymi przejściami.

3.3. Model kart i sterowanie talią

Reprezentacja kart i mechanizm sterowania talią stanowią podstawową warstwę środowiska. Elementy te zostały zaprojektowane niezależnie od zasad Ultimate Texas Hold'em, dzięki czemu mogą być wykorzystywane także przez inne warianty pokera oraz przez moduł oceniający układy.

3.3.1. Reprezentacja karty

Pojedyncza karta jest reprezentowana przez niemodyfikowalny obiekt `Card`, zawierający dwa pola: rangę oraz kolor. Rangi zapisano za pomocą typu wyliczeniowego `Rank`, natomiast kolory za pomocą typu `Suit`. Zastosowanie typów wyliczeniowych ogranicza możliwość utworzenia karty zawierającej niepoprawną wartość.

Rangi kart przyjmują wartości liczbowe od 2 do 14. Karty od dwójki do dziesiątki zachowują swoje naturalne wartości, natomiast walet, dama, król i as otrzymują odpowiednio wartości 11, 12, 13 i 14. Przyjęta reprezentacja umożliwia bezpośrednie porównywanie rang podczas oceny układów pokerowych. W podstawowej reprezentacji as jest kartą najwyższą, natomiast szczególny przypadek strita od asa do piątki obsługiwany jest przez evaluator układów.

Zbiór kolorów obejmuje trefle, karo, kiery i piki. Każdemu kolorowi oraz każdej randze przypisano również symbol tekstowy, umożliwiający czytelne przedstawienie kart, na przykład `A♠` lub `10♥`. Reprezentacja tekstowa jest wykorzystywana głównie w testach, diagnostyce oraz zapisie przebiegu rozdania.

Obiekty kart są niemodyfikowalne po ich utworzeniu. Pozwala to bezpiecznie umieszczać je w krotkach i zbiorach, a także wykorzystywać porównanie liczności zbioru do wykrywania zduplikowanych kart. Konstruktor dodatkowo sprawdza, czy przekazana ranga i kolor należą do odpowiednich typów wyliczeniowych.

3.3.2. Standardowa talia

Standardową talię reprezentuje klasa `Deck`. Podczas jej tworzenia generowane są wszystkie kombinacje trzynastu rang i czterech kolorów. Liczba kart w pełnej talii wynosi zatem

$$|\mathcal{D}| = 13 \cdot 4 = 52. \quad (3.1)$$

Ponieważ każda kombinacja rangi i koloru występuje dokładnie raz, wygenerowana talia zawiera 52 unikalne karty. Karty są przechowywane wewnętrznie w modyfikowalnej kolekcji, jednak publiczny podgląd zawartości talii zwracany jest w postaci krotki. Unie umożliwia to zewnętrzną zmianę kolejności lub usunięcie karty z pominięciem operacji zdefiniowanych przez talię.

Podstawową operacją jest metoda `draw()`, która pobiera jedną kartę z wierzchu

talii i jednocześnie usuwa ją z kolekcji. Dostępna jest również operacja pobrania wielu kart. Próba pobrania karty z pustej talii lub większej liczby kart niż liczba pozostałych elementów powoduje zgłoszenie wyjątku. Takie zachowanie umożliwia szybkie wykrycie niepoprawnej implementacji przebiegu rozdania.

Domyślnie talia jest tasowana bezpośrednio po utworzeniu. Możliwe jest także utworzenie talii w kolejności początkowej, co bywa użyteczne podczas testowania samego modelu kart. Tasowanie wykonywane jest przy użyciu lokalnego generatora liczb pseudolosowych, a nie globalnego stanu losowości programu.

3.3.3. Abstrakcja źródła kart

Silnik rozdania nie zależy bezpośrednio od klasy `Deck`. Zamiast tego korzysta z abstrakcji `CardSource`, która wymaga jedynie udostępnienia operacji `draw()`. Każdy obiekt spełniający ten kontrakt może zostać użyty jako źródło kolejnych kart.

Minimalny interfejs źródła kart oddziela logikę rozdania od sposobu przechowywania i wybierania kart. W normalnej rozgrywce źródłem jest pseudolosowo przetasowana talia, natomiast w testach można przekazać źródło deterministyczne. Moduł odpowiedzialny za rozdawanie wykonuje w obu przypadkach te same operacje i nie musi rozpoznawać rodzaju używanej talii.

3.3.4. Deterministyczna talia testowa

Do testowania konkretnych scenariuszy wprowadzono klasę `FixedDeck`. Podczas jej tworzenia przekazywana jest jawna sekwencja kart określająca kolejność dobierania. Pierwszy element przekazanej sekwencji jest pierwszą kartą zwracaną przez metodę `draw()`.

Przed zapisaniem sekwencji sprawdzane jest, czy wszystkie jej elementy są poprawnymi obiektami `Card` oraz czy żadna karta nie występuje więcej niż raz. Dzięki temu błąd w danych testowych nie może doprowadzić do przeprowadzenia rozdania zawierającego dwie identyczne karty.

Deterministyczne źródło umożliwia przygotowanie testów dla ściśle określonych sytuacji, takich jak:

- wygrana gracza przy niekwalifikującym się krupierze,
- remis obu stron,
- uzyskanie konkretnego układu wypłacanego przez zakład `Blind`,
- sprawdzenie kolejności kart gracza i krupiera,
- weryfikacja kart spalonych przed flopem oraz przed turnem,
- próba wykonania przejścia przy niewystarczającej liczbie kart.

3.3.5. Powtarzalność losowych rozdań

Powtarzalność eksperymentów zapewniono przez możliwość przekazania ziarna generatora liczb pseudolosowych. Utworzenie dwóch talii z tym samym ziarnem prowadzi do uzyskania tej samej kolejności kart, o ile wykonano na nich tę samą sekwencję operacji.

Na poziomie klasy `UTHGame` zastosowano dodatkowy generator nadrzędny. Przy każdym rozpoczęciu nowego rozdania generuje on osobne ziarno przekazywane do nowej talii. W rezultacie kolejne rozdania w obrębie jednego eksperymentu różnią się od siebie, ale cała ich sekwencja może zostać odtworzona przez ponowne uruchomienie środowiska z tym samym ziarnem początkowym.

Rozwiązanie to pozwala porównywać agentów na odpowiadających sobie sekwencjach rozdań. Jeżeli dwa eksperymenty zostaną uruchomione z tym samym ziarnem środowiska i zachowają tę samą kolejność wywołań, otrzymają identyczne strumienie talii. Ogranicza to wpływ losowego doboru kart na ocenę różnic pomiędzy badanymi strategiami.

3.4. Ocena układów pokerowych

Moduł oceny układów pokerowych został zaimplementowany niezależnie od reguł Ultimate Texas Hold'em. Jego zadaniem jest określenie wartości dowolnego poprawnego układu pięciokartowego oraz wybranie najlepszej możliwej ręki z pięciu, sześciu lub siedmiu dostępnych kart. W środowisku UTH evaluator jest wykorzystywany podczas showdownu osobno dla gracza i krupiera.

Hierarchię układów pokerowych oraz ich znaczenie przedstawiono w rozdziale teoretycznym. W niniejszej sekcji opisano sposób odwzorowania tej hierarchii w implementacji oraz mechanizm jednoznacznego porównywania dwóch rąk.

3.4.1. Reprezentacja wartości ręki

Kategorię układu reprezentuje typ wyliczeniowy `HandRank`. Kolejnym kategoriom przypisano rosnące wartości liczbowe, począwszy od wysokiej karty, a kończąc na pokerze. Dzięki temu porównanie kategorii może zostać wykonane przez porównanie odpowiadających im wartości całkowitych.

Sama kategoria nie wystarcza jednak do rozstrzygnięcia wszystkich przypadków. Dwie ręce mogą należeć do tej samej kategorii, lecz różnić się rangą kart tworzących układ albo kart dodatkowych. Z tego względu pełna wartość ręki jest reprezentowana przez obiekt `HandValue`, zawierający:

- kategorię układu `rank`,
- uporządkowaną krotkę wartości rozstrzygających remis `tiebreak`.

Klucz porównawczy ręki można zapisać jako

$$K(h) = (r(h), t_1(h), t_2(h), \dots, t_n(h)), \quad (3.2)$$

gdzie $r(h)$ oznacza wartość kategorii układu, natomiast $t_1(h), \dots, t_n(h)$ są kolejnymi wartościami rozstrzygającymi remis. Dwa klucze porównywane są leksykograficznie. W pierwszej kolejności porównywana jest kategoria układu, a dopiero w przypadku jej równości kolejne elementy krotki `tiebreak`.

Przykładowo wartość pary asów z królem, dziesiątką i dziewiątką jako kartami dodatkowymi może zostać zapisana w postaci

$$K(h) = (\text{ONE_PAIR}, 14, 13, 10, 9). \quad (3.3)$$

Jeżeli druga ręka również zawiera parę asów, porównanie przechodzi kolejno do króla, dziesiątki i dziewiątki. Kolory kart nie uczestniczą w rozstrzyganiu remisów, zgodnie ze standardowymi zasadami pokera.

Obiekt `HandValue` sprawdza zgodność długości krotki `tiebreak` z kategorią układu. Walidowane jest również, czy wszystkie jej elementy są liczbami całkowitymi odpowiadającymi poprawnym rangom kart. Ogranicza to możliwość utworzenia wartości ręki, która nie mogłaby powstać w prawidłowym rozdaniu.

3.4.2. Ocena układu pięciokartowego

Funkcja `evaluate_five_card_hand()` przyjmuje dokładnie pięć unikalnych kart. Przed rozpoczęciem oceny sprawdzana jest liczba kart, typ każdego elementu oraz brak duplikatów.

W pierwszym kroku obliczana jest liczba wystąpień każdej rangi. Na tej podstawie możliwe jest wykrycie par, dwóch par, trójki, fulla oraz karety. Niezależnie sprawdzane są dwa warunki:

- czy wszystkie karty mają ten sam kolor,
- czy pięć różnych rang tworzy ciąg kolejnych wartości.

Kategorie sprawdzane są od najsilniejszej do najslabszej. Jeżeli spełnione są jednocześnie warunki strita i koloru, zwracany jest poker. W przeciwnym razie sprawdzane są kolejno kareta, full, kolor, strit, trójka, dwie pary, jedna para oraz wysoka karta.

Wartość `tiebreak` jest budowana bezpośrednio podczas rozpoznawania kategorii. Wynikiem funkcji nie jest zatem jedynie nazwa układu, lecz kompletna wartość umożliwiająca porównanie z dowolną inną poprawną ręką.

3.4.3. Obsługa niskiego strita z asem

W podstawowym modelu rang as otrzymuje wartość 14 i jest najwyższą kartą. W przypadku strita `A-2-3-4-5` musi jednak pełnić funkcję karty niższej od dwójki. Układ

ten jest obsługiwany jako szczególny przypadek.

Dla zestawu rang

$$\{14, 5, 4, 3, 2\} \quad (3.4)$$

evaluator zwraca strita o najwyższej karcie równej 5. Jego klucz porównawczy jest zatem słabszy od klucza strita zakończonego szóstką. Pozwala to zachować naturalną kolejność wszystkich możliwych stritów bez zmieniania podstawowej wartości asa w pozostałych kategoriach.

3.4.4. Wybór najlepszej ręki

Podczas showdownu gracz i krupier dysponują łącznie siedmioma kartami: dwiema kartami własnymi oraz pięcioma kartami wspólnymi. Rękę pokerową tworzy jednak dokładnie pięć kart. Funkcja `evaluate_best_hand()` generuje wszystkie możliwe pięciokartowe podzbiory dostępnych kart, ocenia każdy z nich, a następnie wybiera układ o największej wartości `HandValue`.

Liczba sprawdzanych kombinacji dla n dostępnych kart wynosi

$$N(n) = \binom{n}{5}. \quad (3.5)$$

Dla liczby kart obsługiwanych przez evaluator otrzymuje się:

$$\binom{5}{5} = 1, \quad \binom{6}{5} = 6, \quad \binom{7}{5} = 21. \quad (3.6)$$

W przypadku standardowego showdownu UTH konieczne jest zatem sprawdzenie 21 kombinacji dla gracza oraz 21 kombinacji dla krupiera. Ze względu na niewielką i stałą liczbę możliwości pełne przeszukanie jest wystarczająco proste, jednoznaczne oraz łatwe do zweryfikowania w testach. Nie było potrzeby stosowania bardziej złożonego, specjalizowanego algorytmu ewaluacji.

Przed wygenerowaniem kombinacji sprawdzane jest, czy przekazano od pięciu do siedmiu kart, czy wszystkie elementy są kartami oraz czy żadna karta nie występuje więcej niż raz.

Wynikiem operacji jest obiekt `EvaluatedHand`, który przechowuje zarówno wartość `HandValue`, jak i dokładnie pięć kart tworzących wybrany układ. Zachowanie kart składowych jest przydatne podczas:

- prezentowania wyniku showdownu,
- analizy decyzji agentów,
- diagnozowania błędów evaluatora,

- identyfikowania rzadkich lub szczególnie wartościowych rozdzeń.

Poker królewski nie stanowi osobnej kategorii `HandRank`. Jest reprezentowany jako poker z asem jako najwyższą kartą. Obiekt `HandValue` udostępnia właściwość pozwalającą rozpoznać ten przypadek, co jest potrzebne przy naliczaniu najwyższej wypłaty zakładu `Blind`.

3.5. Stan wewnętrzny i obserwacja agenta

Ultimate Texas Hold'em jest grą z niepełną informacją. W momencie podejmowania decyzji gracz zna własne karty oraz dotychczas odkryte karty wspólne, ale nie zna kart krupiera, kart spalonych ani kolejności pozostałych kart w talii. Poprawne odwzorowanie tej własności wymaga rozdzielenia pełnego stanu środowiska od informacji udostępnianych agentowi.

W implementacji zastosowano dwa odrębne modele danych:

- `RoundState`, reprezentujący pełny stan wewnętrzny rozdania,
- `UTHObservation`, reprezentujący obserwację dostępną agentowi.

Agent nie otrzymuje bezpośredniego odwołania do obiektu `RoundState`. Każda obserwacja jest tworzona na jego podstawie przez osobną funkcję projekcji, która kopiuje wyłącznie informacje dozwolone z perspektywy gracza.

3.5.1. Pełny stan rozdania

Obiekt `RoundState` zawiera wszystkie dane potrzebne do kontynuowania i rozstrzygnięcia rozdania. Obejmuje on:

- aktualną fazę gry,
- dwie karty własne gracza,
- dwie ukryte karty krupiera,
- odkryte karty wspólne,
- karty spalone,
- mnożnik wykonanego zakładu `Play`,
- końcowy wynik rozdania,
- szczegółowe rozliczenie zakładów,
- rezultat showdownu.

Nie wszystkie pola są aktywne jednocześnie. W stanach pośrednich wynik rozdania, rozliczenie, rezultat showdownu oraz mnożnik zakładu Play pozostają nieokreślone. Pola te otrzymują wartości dopiero po przejściu do stanu terminalnego.

Liczba kart wspólnych i spalonych jest uzależniona od aktualnej fazy. Zależność tę przedstawiono w tabeli 3.2.

Tabela 3.2. Liczba kart przechowywanych w stanie w poszczególnych fazach

Faza	Karty wspólne	Karty spalone	Znaczenie
PREFLOP	0	0	Rozdano wyłącznie karty własne gracza i krupiera.
FLOP	3	1	Spalono jedną kartę i odkryto flop.
RIVER	5	2	Odkryto komplet kart wspólnych.
TERMINAL	5	2	Rozdanie zostało zakończone foldem albo showdownem.

Obiekt stanu jest niemodyfikowalny. Każde przejście środowiska prowadzi do utworzenia nowej instancji RoundState, zamiast modyfikowania istniejącego obiektu. Upraszcza to analizę przejść, ogranicza ryzyko częściowej aktualizacji oraz ułatwia tworzenie deterministycznych testów.

Podczas tworzenia stanu sprawdzane są między innymi:

- poprawny typ fazy,
- obecność dokładnie dwóch kart gracza i dwóch kart krupiera,
- liczba kart wspólnych właściwa dla danej fazy,
- liczba kart spalonych właściwa dla danej fazy,
- brak zduplikowanych kart w całym stanie,
- zgodność pól końcowych z fazą rozdania.

Walidacja obejmuje również zależności semantyczne pomiędzy polami. Stan niekońcowy nie może zawierać wyniku ani rozliczenia. Stan zakończony foldem nie może zawierać mnożnika Play ani rezultatu showdownu. Z kolei stan zakończony showdownem musi zawierać zarówno mnożnik wykonanego zakładu, jak i szczegóły ocenionych rąk.

3.5.2. Obserwacja agenta

Obiekt UTHObservation zawiera wyłącznie dane, które mogą zostać wykorzystane przez agenta podczas podejmowania decyzji. Są to:

- aktualna faza gry,
- dwie karty własne gracza,
- dotychczas odkryte karty wspólne,
- zbiór legalnych akcji,
- mnożnik zakładu Play w obserwacji terminalnej.

Obserwacja celowo nie zawiera:

- kart krupiera,
- kart spalonych,
- pozostałych kart w talii,
- kolejności przyszłego dobierania kart,
- wyniku evaluacji ręki krupiera przed zakończeniem rozdania.

Zbiór legalnych akcji jest umieszczony bezpośrednio w obserwacji. Agent nie musi zatem samodzielnie odtwarzać zasad gry na podstawie fazy. Rozwiązanie to ułatwia implementację agentów bazowych, a w przyszłym adapterze Gymnasium umożliwi utworzenie maski akcji.

Obserwacja, podobnie jak pełny stan, jest obiektem niemodyfikowalnym. Sprawdzana jest liczba widocznych kart, brak duplikatów oraz zgodność zbioru legalnych akcji z aktualną fazą gry. Dzięki temu błędnie skonstruowana obserwacja jest odrzucana przed przekazaniem jej agentowi.

3.5.3. Projekcja stanu na obserwację

Proces tworzenia obserwacji można przedstawić jako funkcję projekcji

$$O_t = \phi(S_t), \quad (3.7)$$

gdzie S_t jest pełnym stanem środowiska w chwili t , natomiast O_t oznacza obserwację udostępnioną agentowi.

Dla zaimplementowanego środowiska projekcja ma postać

$$\phi(S_t) = (p_t, C_t^{player}, C_t^{community}, A_t^{legal}, m_t), \quad (3.8)$$

gdzie:

- p_t oznacza aktualną fazę gry,

- C_t^{player} oznacza karty własne gracza,
- $C_t^{community}$ oznacza odkryte karty wspólne,
- A_t^{legal} oznacza zbiór legalnych akcji,
- m_t oznacza mnożnik zakładu Play, jeżeli został już wykonany.

Elementy takie jak karty krupiera, karty spalone i zawartość pozostałej talii należą do S_t , ale nie są elementami O_t .

Porównanie zawartości pełnego stanu i obserwacji przedstawiono w tabeli 3.3.

Tabela 3.3. Porównanie pełnego stanu środowiska i obserwacji agenta

Informacja	RoundState	UTHObservation
Aktualna faza	tak	tak
Karty własne gracza	tak	tak
Odkryte karty wspólne	tak	tak
Karty krupiera	tak	nie
Karty spalone	tak	nie
Kolejność pozostałej talii	dostępna wewnętrznie silnikowi	nie
Legalne akcje	wyznaczane na podstawie fazy	tak
Mnożnik zakładu Play	tak	tylko po wykonaniu zakładu
Końcowy wynik rozdania	tak	nie
Szczegółowe rozliczenie	tak	nie
Szczegóły showdownu	tak	nie

Wynik rozdania, rozliczenie i szczegóły showdownu są po zakończeniu epizodu zwracane przez obiekt `StepResult`, a nie przez samą obserwację. Informacje te pojawiają się dopiero po wykonaniu ostatniej decyzji, dlatego nie mogą wpłynąć na wybór akcji w bieżącym rozdaniu.

3.5.4. Ochrona przed wyciekami informacji

Rozdzielenie stanu i obserwacji pełni rolę ochrony przed wyciekami informacji ukrytej. Funkcja tworząca obserwację nie usuwa niepożądanych pól z kopii pełnego stanu. Zamiast tego jawnie konstruuje nowy obiekt, przekazując wyłącznie dozwolone wartości.

Podejście to można określić jako tworzenie obserwacji zgodnie z listą dozwolonych informacji. Jest ono bezpieczniejsze niż stosowanie listy pól zabronionych. Po dodaniu nowego pola do `RoundState` nie zostanie ono automatycznie ujawnione agentowi. Aby znalazło się w obserwacji, musi zostać jawnie dodane do modelu `UTHObservation` oraz do funkcji projekcji.

Istotne jest również rozróżnienie pomiędzy interfejsem przeznaczonym dla agenta a interfejsem diagnostycznym. Pełny stan może być wykorzystywany przez testy, rejestrowanie przebiegu rozdania i analizę zakończonych epizodów, ale nie powinien być przekazywany metodzie wybierającej akcję agenta.

Obserwacja nie jest bezpośrednim widokiem pełnego stanu, lecz wynikiem jawnie zdefiniowanej projekcji; informacje ukryte pozostają dostępne wyłącznie wewnętrznym komponentom środowiska.

Informacje ukryte są częścią stanu potrzebnego do poprawnego prowadzenia grywki, lecz nie uczestniczą w tworzeniu obserwacji. Przykładowo silnik musi przechowywać karty krupiera, aby po zakończeniu rozdania przeprowadzić showdown, ale agent nie może korzystać z nich podczas wyboru akcji.

Podobnie karty spalone i kolejność pozostałych kart są potrzebne do kontynuowania rozdania oraz zapewnienia jego deterministycznego odtworzenia, jednak ich ujawnienie zmieniałoby problem decyzyjny. Agent znałby wówczas przyszłe zdarzenia losowe, co prowadziłoby do sztucznie zawyżonej jakości strategii.

Jawna funkcja projekcji stanowi zatem część definicji środowiska z niepełną informacją. Zapewnia ona, że każda metoda wyboru akcji, niezależnie od zastosowanego algorytmu, otrzymuje ten sam zakres informacji. W przyszłych eksperymentach zmianie może podlegać sposób numerycznego kodowania obserwacji, natomiast zbiór informacji dostępnych agentowi pozostanie niezmienny.

3.6. Maszyna stanów i przestrzeń akcji

Przebieg pojedynczego rozdania Ultimate Texas Hold'em odwzorowano jako deterministyczną maszynę stanów. Aktualna faza określa zakres informacji widocznych dla agenta, zbiór legalnych akcji oraz operacje wykonywane przez środowisko po otrzymaniu decyzji. Zbiór faz ma postać

$$\mathcal{P} = \{\text{PREFLOP}, \text{FLOP}, \text{RIVER}, \text{TERMINAL}\}. \quad (3.9)$$

Rozdanie rozpoczyna metoda `reset()`, która rozdaje po dwie karty graczowi i krupierowi w kolejności P_1, D_1, P_2, D_2 , tworzy stan PREFLOP i zwraca pierwszą obserwację. Karty wspólne i spalone nie są jeszcze dobierane. Każde poprawne wykonanie metody `step()` przesuwa rozdanie do kolejnej fazy albo do stanu terminalnego; agent nie może pozostać w tej samej fazie po wykonaniu akcji.

Pełna przestrzeń akcji zawiera sześć decyzji domenowych, a zbiór akcji legalnych zależy od fazy:

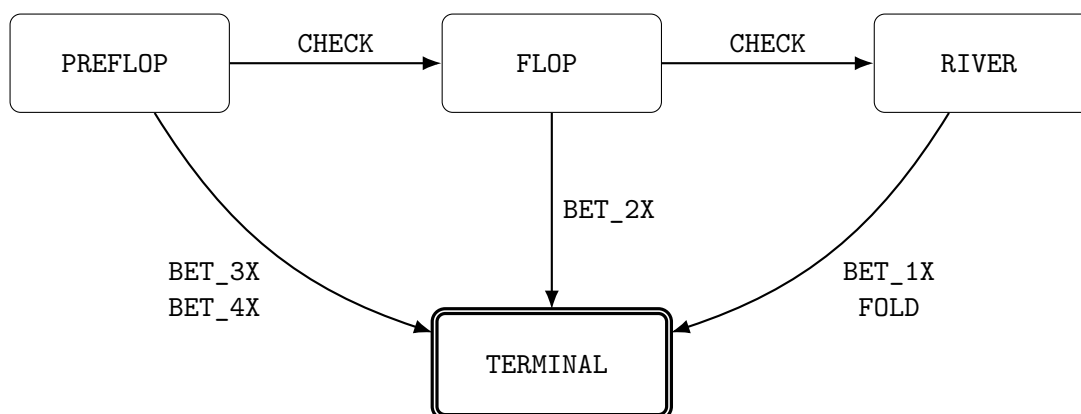
$$\mathcal{A} = \{\text{CHECK}, \text{BET_4X}, \text{BET_3X}, \text{BET_2X}, \text{BET_1X}, \text{FOLD}\}, \quad (3.10)$$

$$\mathcal{A}_{legal}(p) = \begin{cases} \{\text{CHECK}, \text{BET_3X}, \text{BET_4X}\}, & p = \text{PREFLOP}, \\ \{\text{CHECK}, \text{BET_2X}\}, & p = \text{FLOP}, \\ \{\text{BET_1X}, \text{FOLD}\}, & p = \text{RIVER}, \\ \emptyset, & p = \text{TERMINAL}. \end{cases} \quad (3.11)$$

Akcja CHECK odkłada decyzję o zakładzie Play i przesuwa rozdanie do następnej fazy, spalając kartę oraz odkrywając flop albo turn i river. Każda akcja typu BET ustala mnożnik Play wynikający z fazy: $4\times$ lub $3\times$ przed flopem, $2\times$ po flopie i $1\times$ na riverze. Po zakładzie środowisko odkrywa pozostałe karty wspólne, przeprowadza showdown i przechodzi do stanu terminalnego. Akcja FOLD kończy rozdanie na riverze bez showdownu. Zależności te zestawiono w tabeli 3.4 i przedstawiono na rysunku 3.2.

Tabela 3.4. Fazy, legalne akcje i przejścia środowiska UTH

Faza	Legalne akcje	Przejście i operacje
PREFLOP	CHECK, BET_3X, BET_4X	CHECK spala kartę, odkrywa flop i przechodzi do FLOP; zakład BET odkrywa cały stół, prowadzi do showdownu i TERMINAL.
FLOP	CHECK, BET_2X	CHECK odkrywa turn i river i przechodzi do RIVER; BET_2X prowadzi do showdownu i TERMINAL.
RIVER	BET_1X, FOLD	BET_1X prowadzi do showdownu, a FOLD kończy rozdanie bez showdownu; oba przejścia prowadzą do TERMINAL.
TERMINAL	brak	Stan końcowy z wynikiem i rozliczeniem; kolejne rozdanie wymaga wywołania reset().



Rys. 3.2. Maszyna stanów pojedynczego rozdania UTH

3.6.1. Pojedynczy zakład Play i walidacja decyzji

Struktura maszyny stanów gwarantuje, że zakład Play może zostać wykonany najwyżej raz: każda akcja typu BET prowadzi bezpośrednio do stanu terminalnego, a akcje CHECK jedynie przesuwają decyzję do późniejszej fazy. Najdłuższa ścieżka zawiera trzy decyzje

$$\text{PREFLOP} \xrightarrow{\text{CHECK}} \text{FLOP} \xrightarrow{\text{CHECK}} \text{RIVER} \xrightarrow{\text{BET_1X lub FOLD}} \text{TERMINAL}, \quad (3.12)$$

natomiast najkrótsza jedną, po zakładzie $3\times$ lub $4\times$ przed flopem. Zastosowanie osobnych akcji BET_1X, BET_2X, BET_3X i BET_4X zamiast jednej ogólnej akcji BET wiąże mnożnik Play jednoznacznie z fazą i eliminuje możliwość przekazania niepoprawnej wysokości zakładu.

Zbiór legalnych akcji jest wyznaczany przez środowisko i umieszczany w obiekcie UTHObservation, dzięki czemu agent nie musi samodzielnie odtwarzać reguł legalności. Nie oznacza to jednak, że środowisko ufa zwróconej decyzji — każda akcja jest ponownie walidowana przez metodę step(). Przed wykonaniem operacji zmieniającej stan sprawdzane jest kolejno, czy rozdanie zostało rozpoczęte, czy stan nie jest terminalny, czy przekazana wartość jest obiektem typu Action oraz czy należy do zbioru $\mathcal{A}_{\text{legal}}$ dla bieżącej fazy. Naruszenie tych warunków powoduje zgłoszenie odpowiednio RoundNotStartedError, RoundFinishedError albo IllegalActionError.

Walidacja następuje przed dobieraniem kart i zapisaniem nowego stanu, więc nielegalna decyzja nie zużywa kart z talii, nie zmienia fazy rozdania i nie jest dodawana do historii. Środowisko nie zastępuje nielegalnej akcji inną decyzją ani nie nakłada automatycznej kary; obsługa błędów należy do komponentu sterującego eksperymentem.

3.7. Showdown i rozliczanie zakładów

Każde rozdanie, w którym gracz wykonał zakład Play, kończy się showdownem. Środowisko odkrywa wszystkie brakujące karty wspólne, ocenia najlepszą pięciokartową rękę gracza i krupiera, a następnie porównuje otrzymane wartości. Wynik porównania stanowi podstawę do rozliczenia zakładów Ante, Blind oraz Play.

Spasowanie gracza jest obsługiwane oddzielnie. W takim przypadku showdown nie jest przeprowadzany, ponieważ wynik rozdania wynika bezpośrednio z decyzji gracza.

3.7.1. Przebieg showdownu

Podczas showdownu każda strona dysponuje siedmioma kartami:

$$C^{\text{player}} = C_{\text{private}}^{\text{player}} \cup C^{\text{community}}, \quad (3.13)$$

$$C^{\text{dealer}} = C_{\text{private}}^{\text{dealer}} \cup C^{\text{community}}, \quad (3.14)$$

gdzie $C_{private}^{player}$ i $C_{private}^{dealer}$ oznaczają po dwie karty własne, natomiast $C^{community}$ zawiera pięć kart wspólnych.

Evaluator wybiera najlepszy pięciokartowy układ osobno dla gracza i krupiera. Wynik tej operacji jest przechowywany w obiekcie ShowdownResult, który zawiera:

- najlepszą rękę gracza,
- najlepszą rękę krupiera,
- wynik porównania z perspektywy gracza,
- informację o kwalifikacji krupiera.

Wynik showdownu należy do zbioru

$$\mathcal{O}_{showdown} = \{\text{PLAYER_WIN}, \text{DEALER_WIN}, \text{PUSH}\}. \quad (3.15)$$

Dla wartości rąk H_p oraz H_d wynik można zdefiniować jako

$$O(H_p, H_d) = \begin{cases} \text{PLAYER_WIN}, & H_p > H_d, \\ \text{DEALER_WIN}, & H_p < H_d, \\ \text{PUSH}, & H_p = H_d. \end{cases} \quad (3.16)$$

Porównanie rąk jest wykonywane niezależnie od kwalifikacji krupiera. Oznacza to, że krupier może wygrać rozdanie również wtedy, gdy jego ręka nie spełnia warunku kwalifikacji. Przykładem jest sytuacja, w której obie strony mają wyłącznie wysoką kartę, ale wysoka karta krupiera jest silniejsza.

3.7.2. Kwalifikacja krupiera

W podstawowym wariantcie Ultimate Texas Hold'em krupier kwalifikuje się, jeżeli posiada co najmniej jedną parę. Warunek kwalifikacji można zapisać jako

$$Q(H_d) = \begin{cases} 1, & \text{rank}(H_d) \geq \text{ONE_PAIR}, \\ 0, & \text{rank}(H_d) = \text{HIGH_CARD}. \end{cases} \quad (3.17)$$

Kwalifikacja nie zmienia wyniku porównania rąk. Wpływa natomiast na sposób rozliczenia zakładu Ante:

- jeżeli krupier kwalifikuje się, Ante wygrywa lub przegrywa zgodnie z wynikiem showdownu,
- jeżeli krupier nie kwalifikuje się, Ante jest zwracane niezależnie od tego, która strona ma silniejszą rękę.

Zakład Play jest zawsze rozliczany na podstawie bezpośredniego porównania rąk. Zakład Blind jest rozliczany według układu gracza w przypadku jego zwycięstwa, zwracany przy remisie oraz przegrywany przy zwycięstwie krupiera.

Zależności te przedstawiono w tabeli 3.5.

Tabela 3.5. Rozliczenie zakładów po showdownie

Wynik	Kwalifikacja	Ante	Blind	Play
Wygrana gracza	tak	wygrana 1:1	według tabeli wypłat	wygrana 1:1
Wygrana gracza	nie	zwrot	według tabeli wypłat	wygrana 1:1
Remis	dowolna	zwrot	zwrot	zwrot
Wygrana krupiera	tak	przegrana	przegrana	przegrana
Wygrana krupiera	nie	zwrot	przegrana	przegrana

Ostatni przypadek jest istotny z punktu widzenia poprawności implementacji. Brak kwalifikacji krupiera nie oznacza automatycznego zwycięstwa gracza. Jeżeli krupier ma silniejszą wysoką kartę, wygrywa porównanie, zakłady Blind i Play są przegrywane, natomiast Ante zostaje zwrócone.

3.7.3. Model rozliczenia zakładów

Wynik finansowy każdego zakładu reprezentuje obiekt `WagerSettlement`. Zawiera on dwie wartości:

- `stake` – początkową wartość postawionego zakładu,
- `net_profit` – zysk albo stratę netto.

Wszystkie kwoty są wyrażane w jednostkach Ante. Dla pojedynczego zakładu o stawce s i zysku netto n całkowity zwrot brutto wynosi $g = s + n$; wartość $g = s$ oznacza zwrot stawki, $g = 0$ jej pełną utratę, a stawka $s = 0$ zakład niepostawiony.

Pełne rozliczenie rozdania reprezentuje obiekt `Settlement`, zawierający osobne wyniki dla Ante, Blind oraz Play. Umożliwia to analizę zarówno łącznego wyniku rozdania, jak i wpływu poszczególnych składników. Łączny wynik netto jest sumą składników

$$S_{net} = n_{Ante} + n_{Blind} + n_{Play}. \quad (3.18)$$

Wartości finansowe są reprezentowane za pomocą typu `Fraction`. Pozwala to przechowywać wypłaty takie jak $3/2$ bez stosowania przybliżeń zmiennoprzecinkowych. Ma to znaczenie podczas wielokrotnego sumowania wyników w eksperymentach, ponieważ eliminuje błędy zaokrągleń na poziomie pojedynczych rozdań.

3.7.4. Rozliczenie zakładu Blind

Zakład Blind ma stałą stawkę równą jednej jednostce Ante. W przypadku zwycięstwa gracza jego zysk zależy od kategorii uzyskanego układu. Dla układów słabszych od strita Blind jest zwracany. Dla strita lub silniejszego układu stosowana jest tabela wypłat przedstawiona w tabeli 3.6.

Tabela 3.6. Tabela wypłat zakładu Blind

Układ gracza	Wypłata	Zysk netto
Wysoka karta	zwrot	0
Jedna para	zwrot	0
Dwie pary	zwrot	0
Trójka	zwrot	0
Strit	1:1	1
Kolor	3:2	$\frac{3}{2}$
Full	3:1	3
Kareta	10:1	10
Poker	50:1	50
Poker królewski	500:1	500

Poker królewski jest w evaluatorze szczególnym przypadkiem pokera z asem jako najwyższą kartą. Nie stanowi osobnej kategorii układu, ale jest rozpoznawany podczas wyznaczania wypłaty Blind.

Tabela Blind jest stosowana wyłącznie wtedy, gdy gracz wygrał showdown. Przy remisie zakład jest zwracany, natomiast przy zwycięstwie krupiera pełna stawka Blind zostaje utracona.

3.7.5. Rozliczenie Ante i Play

Zakład Ante ma stałą wartość jednej jednostki. Przy remisie jest zwracany. Jeżeli krupier kwalifikuje się, Ante wygrywa albo przegrywa zgodnie z wynikiem showdownu. W przypadku braku kwalifikacji zakład jest zwracany.

Wartość zakładu Play jest określona przez moment podjęcia decyzji:

$$s_{Play} \in \{1, 2, 3, 4\}. \quad (3.19)$$

Zakład Play jest wypłacany 1:1, niezależnie od jego mnożnika. Przykładowo wygrany Play $4\times$ daje zysk netto równy czterem jednostkom. Przy porażce tracona jest pełna wartość zakładu, natomiast przy remisie cała stawka zostaje zwrócona.

3.7.6. Rozliczenie spasowania

Jeżeli gracz spasuje na riverze, nie dochodzi do showdownu. Ante i Blind zostają przegrane, natomiast Play nie został postawiony.

Rozliczenie folda ma zatem postać

$$S_{fold} = (n_{Ante} = -1, n_{Blind} = -1, n_{Play} = 0). \quad (3.20)$$

Łączna stawka wynosi dwie jednostki, a wynik netto jest równy

$$S_{net}^{fold} = -2. \quad (3.21)$$

W rozliczeniu Play zapisywana jest stawka równa zero. Pozwala to odróżnić zakład niepostawiony od postawionego zakładu, który został zwrócony.

3.7.7. Przykładowe rozliczenie

Nietrywialnym przypadkiem jest rozdanie, w którym niekwalifikujący się krupier ma wyższą wysoką kartę niż gracz. Przy zakładzie Play $1 \times$ Ante zostaje zwrócone, natomiast Blind i Play są przegrane, co daje wynik netto

$$S_{net} = 0 - 1 - 1 = -2. \quad (3.22)$$

Przykład ten pokazuje, że warunek kwalifikacji wpływa jedynie na rozliczenie Ante i nie zastępuje porównania układów.

3.8. Funkcja nagrody i wynik epizodu

Mechanizm rozliczania zakładów został oddzielony od funkcji nagrody wykorzystywanej przez algorytmy uczenia przez wzmacnianie. Silnik domenowy wyznacza wyłącznie dokładny wynik finansowy rozdania zgodny z zasadami Ultimate Texas Hold'em, natomiast konstruowanie sygnału nagrody należy do warstwy integrującej silnik z agentem. Dzięki temu ta sama implementacja zasad gry może być wykorzystywana w eksperymentach z różnymi funkcjami nagrody, bez modyfikowania showdownu, kwalifikacji krupiera ani tabel wypłat.

Pojedyncze rozdanie stanowi jeden epizod. W podstawowym wariantcie źródłem nagrody terminalnej jest łączny wynik netto rozdania $S_{net} = n_{Ante} + n_{Blind} + n_{Play}$, a wszystkie przejścia pośrednie otrzymują nagrodę równą zero:

$$r_t = \begin{cases} 0, & s_{t+1} \notin \mathcal{S}_{terminal}, \\ S_{net}, & s_{t+1} \in \mathcal{S}_{terminal}. \end{cases} \quad (3.23)$$

Wynik netto wybrano zamiast zwrotu brutto, ponieważ nie zawiera zwracanych stawek początkowych: zwrócony zakład otrzymuje wtedy wartość zero. Wartości rozliczeń są przechowywane dokładnie za pomocą typu `Fraction`, a adapter środowiska przekształca końcową wartość na typ liczbowy wymagany przez bibliotekę uczenia dopiero na granicy interfejsu.

Alternatywne funkcje nagrody, na przykład znormalizowane względem liczby postawionych jednostek albo oparte wyłącznie na znaku wyniku, zostaną przedstawione w rozdziale poświęconym metodyce eksperymentów. Podstawowym punktem odniesienia pozostanie nagroda terminalna równa wynikowi netto rozdania.

3.9. Interfejs silnika

Silnik domenowy udostępnia niewielki interfejs publiczny, za pomocą którego można rozpocząć rozdanie, pobrać obserwację, wykonać decyzję oraz odczytać wynik zakończonego epizodu. Interfejs nie zależy od konkretnego algorytmu uczenia ani od biblioteki Gymnasium.

Centralnym elementem jest klasa `UTHGame`. Pojedyncza instancja przechowuje stan aktualnego rozdania, źródło kart oraz opcjonalną historię decyzji. Po zakończeniu rozdania ta sama instancja może zostać wykorzystana ponownie przez wywołanie metody `reset()`.

3.9.1. Tworzenie silnika

Konstruktor klasy `UTHGame` przyjmuje dwa opcjonalne parametry:

- `seed` – ziarno nadrzędnego generatora liczb pseudolosowych,
- `record_history` – informację, czy należy rejestrować przebieg kolejnych decyzji.

Ziarno określa powtarzalny strumień talii generowanych dla kolejnych rozdawań. Parametr nie ustala pojedynczej, niezmienniej talii dla całej instancji. Każde wywołanie `reset()` tworzy nową talię, której ziarno jest pobierane z generatora środowiska.

Rejestrowanie historii jest domyślnie wyłączone. Dzięki temu podstawowe wykonywanie dużej liczby rozdań nie wymaga przechowywania dodatkowych obiektów diagnostycznych.

3.9.2. Rozpoczęcie epizodu

Metoda `reset()` rozpoczyna nowe rozdanie i zwraca pierwszą obserwację w fazie PREFLOP. W standardowym użyciu środowisko samodzielnie tworzy przetasowaną talię. Metoda umożliwia również przekazanie własnego obiektu implementującego interfejs `CardSource`, na przykład deterministycznej talii `FixedDeck`.

Sygnaturę operacji można przedstawić jako

$$reset : CardSource \cup \{None\} \rightarrow UTHObservation. \quad (3.24)$$

Rozpoczęcie rozdania obejmuje:

1. wybór lub utworzenie źródła kart,

2. rozdanie dwóch kart graczowi i dwóch kart krupierowi,
3. utworzenie stanu PREFLOP,
4. wyzerowanie historii decyzji bieżącego rozdania,
5. utworzenie obserwacji gracza.

Reset jest wykonywany transakcyjnie. Silnik najpierw próbuje pobrać wszystkie karty potrzebne do utworzenia stanu początkowego, a dopiero po powodzeniu zapisuje nowy stan i źródło kart. Jeżeli przekazane źródło nie może dostarczyć wymaganych kart, poprzedni stan silnika nie zostaje zastąpiony stanem częściowo utworzonego rozdania.

3.9.3. Wykonanie decyzji

Metoda `step()` przyjmuje jedną wartość typu `Action` i wykonuje odpowiadającą jej przejście. Jej uproszczona sygnatura ma postać

$$\text{step} : \text{Action} \rightarrow \text{StepResult}. \quad (3.25)$$

Wynikiem operacji jest obiekt `StepResult`, zawierający:

- obserwację po wykonaniu akcji,
- wynik rozdania, jeżeli osiągnięto stan terminalny,
- rozliczenie zakładów w stanie terminalnym,
- szczegóły showdownu, jeżeli rozdanie nie zakończyło się foldem.

Wynik kroku pośredniego zawiera jedynie kolejną obserwację. Pola dotyczące wyniku i rozliczenia pozostają wtedy nieokreślone. W kroku terminalnym obiekt zawiera komplet informacji niezbędnych do obliczenia nagrody i zarejestrowania wyniku eksperymentu.

Właściwość `terminated` obiektu `StepResult` informuje, czy wykonana akcja zakończyła rozdanie. Jej wartość jest wyznaczana na podstawie fazy zawartej w obserwacji.

3.9.4. Właściwości publiczne

Poza metodami `reset()` i `step()` silnik udostępnia właściwości przedstawione w tabeli 3.7.

Właściwość `state` nie jest częścią interfejsu przeznaczonego do podejmowania decyzji przez agenta. Udostępniono ją na potrzeby testów, diagnostyki i analizy środowiska. Agent powinien otrzymywać wyłącznie wartość właściwości `observation` albo obserwację zwróconą przez `reset()` lub `step()`.

Tabela 3.7. Publiczny interfejs klasy UTHGame

Element	Typ wyniku	Znaczenie
reset()	UTHObservation	Rozpoczyna nowe rozdanie i zwraca obserwację preflop.
step(action)	StepResult	Wykonuje legalną akcję i zwraca wynik przejścia.
observation	UTHObservation	Zwraca aktualne informacje widoczne dla agenta.
state	RoundState	Zwraca pełny stan wewnętrzny, przeznaczony głównie do testów i diagnostyki.
is_started	bool	Informuje, czy rozpoczęto co najmniej jedno rozdanie.
history_enabled	bool	Informuje, czy rejestrowanie historii jest aktywne.
trace	RoundTrace lub None	Zwraca zapis bieżącego rozdania, jeżeli historia jest włączona.

3.9.5. Integracja z Gymnasium

Silnik UTHGame jest niezależny od biblioteki Gymnasium. Integracja z algorytmami uczenia przez wzmocnienie zostanie zrealizowana przez osobny adapter, który nie będzie implementował zasad gry, lecz jedynie warstwę translacyjną: numeryczne kodowanie obserwacji, odwzorowanie indeksów akcji na wartości Action, utworzenie maski legalnych akcji oraz wyznaczenie nagrody i flag terminated i truncated. Wszystkie przejścia, walidacja akcji, showdown i rozliczenia pozostaną odpowiedzialnością silnika domenowego, co zapobiega powstaniu dwóch niezależnych implementacji UTH.

3.10. Rejestrowanie przebiegu i walidacja środowiska

Poprawność środowiska ma bezpośredni wpływ na wyniki eksperymentów. Błąd w kolejności rozdawania kart, rozliczaniu zakładów albo udostępnianiu obserwacji mógłby zostać błędnie zinterpretowany jako zachowanie wyuczone przez agenta. Z tego względu implementację uzupełniono o opcjonalny mechanizm rejestrowania przebiegu rozdania oraz zestaw testów weryfikujących zarówno pojedyncze komponenty, jak i pełne ścieżki rozgrywki.

3.10.1. Opcjonalne rejestrowanie historii

Rejestrowanie historii jest aktywowane parametrem record_history przekazywanym podczas tworzenia obiektu UTHGame. Domyślnie mechanizm pozostaje wyłączony, aby standardowe wykonywanie dużej liczby epizodów nie wymagało tworzenia dodatkowych obiektów diagnostycznych.

Po włączeniu historii każde poprawnie rozpoczęte rozdanie otrzymuje dodatni,

rosnący identyfikator `round_id`. Lista decyzji jest zerowana przy każdym poprawnym wywołaniu metody `reset()`.

Właściwość `trace` zwraca obiekt `RoundTrace`. Jeżeli historia jest wyłączona albo nie rozpoczęto jeszcze rozdania, jej wartością jest `None`.

Mechanizm historii nie stanowi pamięci agenta i nie jest częścią obserwacji. Jest przeznaczony do:

- diagnozowania nieoczekiwanych decyzji,
- odtwarzania przebiegu wybranych rozdań,
- analizy działania agentów bazowych i uczących się,
- zapisywania przykładów błędów podczas eksperymentów,
- weryfikowania zgodności decyzji z legalną przestrzenią akcji.

3.10.2. Rekord pojedynczej decyzji

Pojedynczą decyzję reprezentuje niemodyfikowalny obiekt `DecisionRecord`. Zawiera on:

- obserwację dostępną agentowi przed wykonaniem decyzji,
- wybraną akcję.

Rekord można przedstawić jako parę

$$d_t = (O_t, A_t), \quad (3.26)$$

gdzie O_t oznacza obserwację przed wykonaniem akcji, a A_t oznacza decyzję agenta.

Zapisanie obserwacji sprzed przejścia jest istotne, ponieważ pozwala odtworzyć informacje, na podstawie których agent rzeczywiście podjął decyzję. Zapis obserwacji po wykonaniu akcji prowadziłby do niejednoznaczności: mógłby zawierać karty odkryte dopiero jako skutek tej decyzji.

Podczas tworzenia rekordu sprawdzane jest, czy:

- obserwacja jest obiektem `UTHObservation`,
- akcja jest wartością typu `Action`,
- obserwacja nie jest terminalna,
- akcja należy do zbioru legalnego dla zapisanej obserwacji.

Dzięki temu historia nie może zawierać decyzji wykonanej po zakończeniu rozdania ani akcji sprzecznej z fazą zapisaną w obserwacji.

3.10.3. Ślad całego rozdania

Obiekt RoundTrace reprezentuje niemodyfikowalny zapis aktualnego przebiegu rozdania. Zawiera:

- dodatni identyfikator rozdania,
- uporządkowaną krotkę obiektów DecisionRecord,
- aktualny pełny stan RoundState.

Dla rozdania zawierającego n decyzji jego ślad można zapisać jako

$$\tau = (id, d_0, d_1, \dots, d_{n-1}, S_n). \quad (3.27)$$

Obserwacje znajdujące się w rekordach decyzji zawierają wyłącznie informacje dostępne agentowi. Końcowy element S_n jest natomiast pełnym stanem diagnostycznym i może zawierać karty krupiera, karty spalone oraz rozliczenie rozdania.

Z tego powodu RoundTrace nie może być przekazywany agentowi jako obserwacja. Jest to obiekt przeznaczony dla kodu eksperymentalnego, testów i narzędzi analitycznych.

Ślad udostępnia również właściwości pomocnicze:

- `completed` – informację, czy osiągnięto stan terminalny,
- `outcome` – końcowy wynik rozdania,
- `settlement` – rozliczenie Ante, Blind i Play,
- `showdown` – szczegóły porównania rąk, jeżeli do niego doszło.

Przed zakończeniem rozdania właściwości związane z wynikiem mogą zwracać wartość `None`. Po foldzie dostępne jest rozliczenie i wynik, ale nie występują szczegóły showdownu.

3.10.4. Transakcyjne zapisywanie decyzji

Metoda `step()` najpierw zapisuje bieżącą obserwację w zmiennej lokalnej, następnie próbuje wykonać właściwe przejście, a dopiero po jego powodzeniu dodaje rekord do historii.

Kolejność operacji ma postać

$$O_t \rightarrow \text{validate}(A_t) \rightarrow \text{transition}(S_t, A_t) \rightarrow S_{t+1} \rightarrow \text{record}(O_t, A_t). \quad (3.28)$$

Jeżeli walidacja lub wykonywanie przejścia zakończy się wyjątkiem, etap `record` nie zostaje osiągnięty. Historia zawiera zatem wyłącznie decyzje, dla których środowisko utworzyło poprawny kolejny stan.

Takie zachowanie zabezpiecza między innymi przed zapisaniem:

- akcji nielegalnej w aktualnej fazie,
- decyzji po zakończeniu rozdania,
- akcji, podczas której źródło kart zwróciło niepoprawną wartość,
- niedokończonego przejścia wynikającego z pustej talii.

Ta sama zasada jest stosowana podczas resetowania środowiska. Nowy stan, źródło kart, identyfikator rozdania i pusta historia są zapisywane dopiero po poprawnym rozdaniu wszystkich czterech kart początkowych.

3.10.5. Testy deterministyczne

Znaczna część testów pełnych rozdań wykorzystuje deterministyczne źródło `FixedDeck`. Jawne określenie kolejności kart pozwala przygotować scenariusz, którego wynik nie zależy od generatora pseudolosowego.

Deterministyczny scenariusz może określać kolejno:

$$P_1, D_1, P_2, D_2, B_1, F_1, F_2, F_3, B_2, T, R, \quad (3.29)$$

gdzie P i D oznaczają karty własne gracza i krupiera, B karty spalone, F karty flopa, T turn, a R river.

Tak przygotowane źródło umożliwia kontrolowanie jednocześnie:

- kart obu stron,
- końcowego układu gracza i krupiera,
- kwalifikacji krupiera,
- wyniku showdownu,
- wypłaty zakładu Blind,
- łącznego wyniku netto.

W przeciwieństwie do testu opartego wyłącznie na ziarnie `FixedDeck` jednoznacznie pokazuje w kodzie testowym, jakie karty mają zostać dobrane. Ułatwia to analizę nieudanego przypadku i ogranicza zależność testu od szczegółów algorytmu tasowania.

3.10.6. Poziomy testowania

Testy podzielono według odpowiedzialności komponentów. Testy jednostkowe sprawdzają w izolacji model kart i talię, evaluator, modele stanu, rozdawanie, legalność akcji, showdown oraz rozliczenia. Testy integracyjne wykonują kompletne rozdania obejmujące wszystkie ścieżki maszyny stanów, od `reset()` do stanu terminalnego, wraz z rejestrowaniem historii. Samo poprawne przetestowanie evaluatora i modułu rozliczeń nie gwarantuje bowiem, że silnik przekaże im właściwe karty i mnożnik zakładu Play.

Poza przykładowymi wynikami testy weryfikują niezmienniki obowiązujące dla wszystkich poprawnych rozdań, między innymi brak powtarzających się kart, zgodność liczby kart wspólnych i spalonych z fazą, brak danych terminalnych w stanie pośrednim, brak showdownu po spasowaniu, wykonanie co najwyżej jednego zakładu Play, brak kart krupiera i kart spalonych w obserwacji oraz brak zmiany stanu po odrzuconej akcji. Walidacja obejmuje zatem także przypadki brzegowe i próby utworzenia stanów naruszających reguły domenowe.

3.10.7. Pokrycie kodu

Zestaw testów uruchamiany jest za pomocą biblioteki `pytest`. Podczas prac nad silnikiem analizowano zarówno pokrycie instrukcji, jak i pokrycie rozgałęzień. Pozwoliło to wykryć nieprzetestowane ścieżki walidacji, przypadki brzegowe evaluatora oraz alternatywne sposoby rozliczania zakładów.

Na etapie zakończenia implementacji podstawowego silnika uzyskano pełne pokrycie instrukcji i rozgałęzień dla objętych pomiarem modułów. Wynik pokrycia nie stanowi samodzielnego dowodu poprawności implementacji. Informuje jedynie, że testy wykonały wszystkie mierzone linie i gałęzie programu. Z tego względu został uzupełniony testami opartymi na konkretnych scenariuszach domenowych oraz walidacją niezmienników.

3.11. Ograniczenia środowiska

Zaimplementowane środowisko odwzorowuje podstawowy wariant Ultimate Texas Hold'em potrzebny do prowadzenia eksperymentów nad strategiami podejmowania decyzji. Celem silnika nie jest pełna symulacja działalności stołu kasynowego, lecz dostarczenie jednoznacznego, deterministycznie testowalnego modelu procesu decyzyjnego gracza.

Zakres implementacji został celowo ograniczony do elementów wpływających na decyzje Play i końcowy wynik podstawowych zakładów. Pozostałe mechanizmy mogą zostać dodane w przyszłości jako osobne rozszerzenia, bez zmieniania głównej maszyny stanów.

3.11.1. Zakres podstawowego środowiska

Zaimplementowany wariant środowiska obejmuje:

- jednego gracza rozgrywającego rozdanie przeciwko krupierowi,
- standardową talię 52 kart bez jokerów,
- zakłady Ante, Blind i Play,
- decyzje Play wykonywane przed flopem, po flopie i po odkryciu całego stołu,
- kwalifikację krupiera od pary,
- standardową tabelę wypłat zakładu Blind,
- zakończenie przez showdown albo fold,
- dokładne rozliczenie finansowe w jednostkach Ante,
- kontrolowane źródło losowości i deterministyczne talie testowe,
- opcjonalne rejestrowanie przebiegu rozdania.

Poza jego zakresem pozostają natomiast zakłady boczne Trips i Progressive, model wielu graczy przy stole, saldo gracza i limity stołu, interfejs graficzny, a także adapter Gymnasium oraz numeryczne kodowanie obserwacji, które zostaną dodane jako osobne warstwy.

3.11.2. Pominięcie zakładów dodatkowych

Zakład Trips jest rozliczany na podstawie końcowego układu gracza niezależnie od wyniku porównania z krupierem. Nie wprowadza kolejnego momentu decyzyjnego podczas rozdania, ponieważ jego wartość jest ustalana przed rozdaniem kart. Wpływa jednak na rozkład końcowych wypłat.

Uwzględnienie Trips w podstawowej wersji środowiska utrudniałoby interpretację wyników strategii Play. Agent mógłby uzyskiwać wysokie lub niskie wyniki wynikające z zakładu, na który nie ma wpływu podczas właściwej sekwencji decyzyjnej. Z tego względu bazowe eksperymenty będą oceniały wyłącznie wynik Ante, Blind i Play.

Progressive również nie zmienia późniejszych decyzji Play, ale wymaga dodatkowego modelu wypłat. W wariacie z narastającym jackpotem wartość potencjalnej nagrody zależy ponadto od stanu utrzymywanego pomiędzy rozdaniem. Oznaczałoby to, że epizody nie byłyby w pełni niezależne bez dodatkowego modelowania wartości puli.

Oba zakłady mogą zostać w przyszłości dodane jako rozszerzenia warstwy rozliczeń. Nie wymagają zmiany podstawowej kolejności faz PREFLOP, FLOP, RIVER i TERMINAL, ale zmieniają końcową funkcję wypłaty.

3.11.3. Brak modelu salda i sesji

Silnik traktuje każde rozdanie jako niezależny epizod. Nie przechowuje salda gracza, historii wyników pomiędzy rozdaniem ani ograniczeń wynikających z dostępnego kapitału.

Ante i Blind mają stałą wartość jednej jednostki, natomiast Play jest wielokrotnością Ante wynikającą bezpośrednio z wybranej akcji. Dzięki temu wyniki różnych strategii mogą być porównywane niezależnie od przyjętego nominału pieniężnego.

Brak modelu salda oznacza, że środowisko nie bada między innymi:

- ryzyka bankructwa,
- doboru wysokości podstawowej stawki,
- zarządzania kapitałem pomiędzy rozdaniem,
- warunków zakończenia całej sesji,
- wpływu limitów stołu.

Zagadnienia te dotyczą zarządzania kapitałem, a nie samej strategii decyzji Play. Mogłyby zostać dodane jako zewnętrzna warstwa zarządzająca serią epizodów.

3.11.4. Model pojedynczego gracza

Środowisko reprezentuje jednego gracza oraz jednego krupiera. Nie symuluje innych miejsc przy stole ani kart rozdanych pozostałym uczestnikom.

W rzeczywistej grze obecność innych graczy wpływa na liczbę kart zużytych z talii, ale zakryte karty tych osób pozostają nieznane badanemu graczowi. Modelowanie pełnego stołu zwiększałoby złożoność procesu rozdawania bez zmiany informacji dostępnych agentowi w podstawowym problemie decyzyjnym.

Przyjęte uproszczenie oznacza, że wszystkie niewidoczne karty poza kartami krupiera, kartami spalonymi i odkrytym stołem pozostają w źródle kart. Rozkład kart widocznych dla pojedynczego gracza jest dzięki temu zgodny z losowaniem bez zwracania ze standardowej talii.

3.11.5. Ograniczenie do jednej konfiguracji reguł

Obecna implementacja wykorzystuje jedną konfigurację zasad:

- krupier kwalifikuje się od jednej pary,
- Ante i Blind mają wartość jednej jednostki,
- dostępne mnożniki Play wynoszą 1, 2, 3 i 4,

- stosowana jest jedna tabela wypłat Blind.

Niektóre kasyna lub materiały źródłowe mogą stosować inne tabele wypłat zakładów dodatkowych albo lokalne warianty zasad. W aktualnym silniku wartości podstawowej konfiguracji są jawnie zapisane w module reguł i rozliczeń.

Możliwym rozszerzeniem jest wprowadzenie niemodyfikowalnego obiektu konfiguracyjnego przekazywanego do silnika. Pozwoliłby on wybierać tabelę Blind, warunek kwalifikacji oraz aktywne zakłady bez modyfikowania kodu mechanizmu rozliczeń. Rozszerzenie to nie jest konieczne dla eksperymentów prowadzonych w jednej, wcześniej ustalonej konfiguracji.

3.11.6. Brak numerycznej reprezentacji obserwacji

Obecna obserwacja jest obiektem domenowym zawierającym karty, fazę oraz zbiór legalnych akcji. Taka postać jest czytelna, jednoznaczna i wygodna podczas testowania, ale nie może zostać bezpośrednio przekazana do większości modeli uczenia maszynowego.

Przed rozpoczęciem uczenia konieczne będzie utworzenie encoderów przekształcających `UTHObservation` do wektorów lub struktur numerycznych o stałym rozmiarze. Sposób kodowania nie zostanie umieszczony w klasie `UTHGame`, ponieważ porównanie reprezentacji stanu stanowi jeden z elementów badawczych pracy.

Oddzielenie encodera od silnika umożliwi wykorzystanie identycznych rozdań i reguł przy porównywaniu różnych reprezentacji. Różnice w wynikach nie będą wówczas wynikały z odmiennego działania mechanizmu gry.

3.12. Podsumowanie

W rozdziale przedstawiono projekt i implementację środowiska pojedynczego rozdania Ultimate Texas Hold'em. Silnik został podzielony na niezależne komponenty odpowiedzialne za reprezentację kart, ocenę układów, rozdawanie, przechowywanie stanu, walidację akcji, przeprowadzanie showdownu oraz rozliczanie zakładów.

Przebieg rozdania odwzorowano jako maszynę stanów obejmującą fazy PREFLOP, FLOP, RIVER i TERMINAL. Zbiór legalnych akcji zależy od bieżącej fazy, a każda akcja typu BET kończy część decyzyjną rozdania. Konstrukcja ta gwarantuje, że zakład Play zostanie wykonany najwyżej jeden raz.

Istotnym elementem projektu jest rozdzielenie pełnego stanu `RoundState` od obserwacji `UTHObservation`. Agent otrzymuje wyłącznie własne karty, odkryte karty wspólne, aktualną fazę oraz legalne akcje. Karty krupiera, karty spalone i kolejność pozostałej talii pozostają wewnętrznymi informacjami środowiska. Zapobiega to wykorzystaniu przez agenta informacji niedostępnych graczowi w rzeczywistym rozdaniu.

Showdown wykorzystuje ogólny evaluator pokerowy wybierający najlepsze pięć kart z siedmiu. Rozliczenie Ante, Blind i Play jest wykonywane na podstawie wyniku porównania, kwalifikacji krupiera, mnożnika Play oraz kategorii ręki gracza. Wszystkie

wartości finansowe są przechowywane dokładnie za pomocą typu `Fraction`.

Silnik umożliwia zarówno losowe, jak i deterministyczne rozdawanie kart. Wstrzykiwane źródło `FixedDeck` pozwala tworzyć powtarzalne scenariusze testowe obejmujące pełną kolejność kart. Opcjonalny mechanizm historii zapisuje obserwacje widoczne przed decyzjami oraz końcowy pełny stan diagnostyczny.

Zaimplementowane środowisko stanowi warstwę domenową projektu. Nie zawiera jeszcze numerycznego kodowania obserwacji, funkcji nagrody zwracanej przez interfejs uczenia ani adaptera `Gymnasium`. Elementy te zostaną utworzone jako osobne warstwy, dzięki czemu możliwe będzie porównywanie reprezentacji stanu, funkcji nagrody i algorytmów przy zachowaniu identycznych zasad gry.

Gotowy silnik zapewnia podstawę do implementacji agentów bazowych, infrastruktury eksperymentalnej oraz metod uczenia przez wzmacnianie analizowanych w dalszej części pracy.

Spis rysunków

3.1	Architektura i przepływ informacji w środowisku UTH	20
3.2	Maszyna stanów pojedynczego rozdania UTH	31

Spis tabel

2.1	Ranking układów pokerowych od najsilniejszego do naj słabszego	10
2.2	Porównanie Texas Hold'em i Ultimate Texas Hold'em	11
2.3	Dostępne decyzje gracza w Ultimate Texas Hold'em	12
2.4	Tabela wypłat zakładu Trips przyjęta w modelu środowiska	13
2.5	Przykładowa tabela wypłat zakładu Trips w Ultimate Texas Hold'em . . .	13
2.6	Przykładowe wartości przewagi kasyna dla wybranych gier kasynowych . .	14
3.1	Podział odpowiedzialności pomiędzy modułami środowiska UTH	19
3.2	Liczba kart przechowywanych w stanie w poszczególnych fazach	27
3.3	Porównanie pełnego stanu środowiska i obserwacji agenta	29
3.4	Fazy, legalne akcje i przejścia środowiska UTH	31
3.5	Rozliczenie zakładów po showdownie	34
3.6	Tabela wypłat zakładu Blind	35
3.7	Publiczny interfejs klasy UTHGame	39

Bibliografia

- [1] Tulalip Resort Casino, *Ultimate Texas Hold'em Game Rules*, https://www.everythingtulalip.com/wp-content/uploads/2024/04/ONEclub_0920_GameRules_RackCard_UltimateTexasHoldem-WEB.pdf, Dostęp: 2026-07-12.
- [2] Nevada Gaming Control Board, *Ultimate Texas Hold'em - Rules of Play*, https://www.gaming.nv.gov/siteassets/content/divisions/enforcement/rules-of-play/Ultimate_Texas_Hold_em.pdf, Dostęp: 2026-07-07.
- [3] W. of Odds, *Ultimate Texas Hold'em*, <https://wizardofodds.com/games/ultimate-texas-hold-em/>, Dostęp: 2026-07-07.
- [4] J. B. Maverick, *Understanding the House Edge: How Casinos Ensure Profit*, <https://www.investopedia.com/articles/personal-finance/110415/why-does-house-always-win-look-casino-profitability.asp>, Dostęp: 2026-07-12.
- [5] Wizard of Odds, *What is the House Edge?* <https://wizardofodds.com/gambling/house-edge/>, Dostęp: 2026-07-12.